Log Barrier and Interior Point Methods

CVXPY + IPOPT

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

The Problem We Are Solving

For the rest of this course, the master problem is the **general constrained nonlinear program (NLP)**:

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) \le 0, \quad i = 1,\dots,m \\ & h_j(\mathbf{x}) = 0, \quad j = 1,\dots,p \end{aligned}$$

Standing assumptions (unless stated otherwise):

- f, g_i, h_j are twice continuously differentiable (C^2).
- A constraint qualification holds at any solution (e.g. LICQ or MFCQ).
- The feasible set has nonempty *strict interior*: $\exists\, \mathbf{x}$ with $g_i(\mathbf{x}) < 0$ for all i.

Our goal today. We already know *what* a solution looks like — a KKT point. We now want an *algorithm* that actually *finds* one, even when m, n are large.

Convexity of f and g_i is *not* assumed. IPOPT handles nonconvex NLPs — it just loses the global-optimality guarantee in that case.

Where We Are

Already covered: unconstrained methods (steepest descent, Newton, CG, BFGS, L-BFGS), line search, trust regions, KKT conditions, Lagrangian duality.

The gap: we know *when* a point is optimal (KKT), but how do we *find* it for a large constrained problem?

Today: the barrier idea — convert inequalities into a smooth objective penalty, trace the central path, run Newton on a perturbed KKT system. This is exactly what **IPOPT** does.

References for today. Nocedal & Wright Ch. 19; Biegler Ch. 10–11; Wächter's lecture notes (IPOPT creator); Luenberger & Ye Ch. 15.

Today's roadmap — one long lecture, three acts, four coding breaks

Act I — The convex story: barriers, cones, CVXPY.

- Why KKT is not an algorithm; log barrier in 1D and 2D; central path.
- *Self-concordance* — why log barriers provably work.
- *The cone zoo* — LP, SOCP, SDP, exp, power barriers.
- **Coding Break #1**: CVXPY in practice.
- *DCP + conic lifting* — how CVXPY turns your code into a cone program.
- **Coding Break #2**: DCP as a proof system.

Act II — Primal-dual machinery for general NLP.

- Primal-dual Newton system, step acceptance, convergence theory.
- **Coding Break #3**: Pyomo + IPOPT on the same 2D LP.

Act III — When convexity is gone: IPOPT's engineering layer.

Roadmap

The KKT system is a combinatorial headache

Given $\min f(\mathbf{x})$ s.t. $g_i(\mathbf{x}) \le 0$, $h_j(\mathbf{x}) = 0$:

$$\nabla f + \sum_i s_i^\star \nabla g_i + \sum_j y_j^\star \nabla h_j = 0,$$

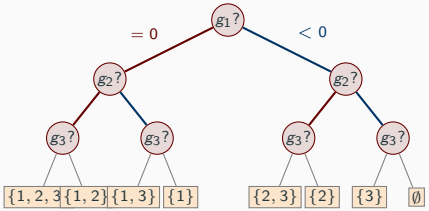
$$h_j = 0, \quad g_i \le 0, \quad s_i^\star \ge 0,$$

$$s_i^\star \, g_i(\mathbf{x}^\star) = 0 \quad \text{(complementary slackness)}.$$

Either $s_i = 0$ or $g_i(x) = 0$ for each i. For m inequalities, 2^m *active sets*.

Active-set methods (SQP, simplex) enumerate cleverly; scale poorly for large m.

Interior point methods take a different route: *smooth out the complementarity*. No enumeration.



$m = 3 \Rightarrow 2^3 = 8$ cases. red $= g_i$ active, blue $= g_i$ slack.

2D KKT in full: $\min x_1 + x_2$ s.t. $x_1^2 + x_2^2 \le 1$

A single inequality, $g(\mathbf{x}) = x_1^2 + x_2^2 - 1 \le 0$. KKT conditions write out completely:

$$\text{(stat) } \nabla f + s\nabla g = \begin{pmatrix} 1 \\ 1 \end{pmatrix} + s \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} = \mathbf{0}, \qquad \text{(feas) } x_1^2 + x_2^2 \le 1, \qquad \text{(dual feas) } s \ge 0,$$

$$\text{(compl) } s\,(x_1^2 + x_2^2 - 1) = 0.$$

Enumerate the $2^1 = 2$ active sets.

Case 1: $s = 0$ (constraint slack). Stationarity becomes $(1, 1) = \mathbf{0}$. Contradiction — no solution.

Case 2: $s > 0$ (constraint active). Then $x_1^2 + x_2^2 = 1$ and from stationarity $x_1 = x_2 = -1/(2s)$. Plug in: $2/(4s^2) = 1 \Rightarrow s = 1/\sqrt{2}$. So $x_1 = x_2 = -1/\sqrt{2}$.

Unique KKT point: $\mathbf{x}^\star = (-1/\sqrt{2},\ -1/\sqrt{2})$, $s^\star = 1/\sqrt{2}$, $f^\star = -\sqrt{2}$. ✓

What you just did by hand. Enumerated $2^m = 2$ cases, solved each system, ruled out the inconsistent one. Easy here; for $m = 30$ it's a billion cases. IPM smooths the complementarity $s \cdot (-g) = \mu$ and lets Newton do it.

The one-dimensional caricature

Consider $\min_x \; x$ s.t. $x \geq 0$.

KKT: stationarity $1 - s = 0$, feasibility $x \geq 0$, complementarity $sx = 0$. Optimum: $x^\star = 0,\ s^\star = 1$.

The barrier idea. Replace the hard constraint $x \geq 0$ with a *penalty* that blows up as $x \to 0^+$:

$$\min_x \; x \; - \; \mu \log x, \qquad \mu > 0.$$

Stationarity: $1 - \mu/x = 0 \;\Rightarrow\; x^\star(\mu) = \mu$.

- Each $\mu > 0$ gives a *strictly interior* minimizer.
- As $\mu \downarrow 0$, $x^\star(\mu) \to 0 = x^\star$.
- Define dual variable $s(\mu) := \mu / x^\star(\mu) = 1$. Complementarity $s \cdot x = \mu$ — perturbed, not zero.

Two ideas in one slide. (1) A *smooth* unconstrained problem per μ. (2) Complementarity becomes $s \cdot x = \mu$, tunable via μ.

Roadmap

The log barrier function

For inequalities $g_i(\mathbf{x}) \le 0,\ i = 1, \dots, m$, the **log barrier** is

$$\Phi(\mathbf{x}) = -\sum_{i=1}^{m} \log(-g_i(\mathbf{x})),$$

defined on the **strict interior** $\{\mathbf{x} : g_i(\mathbf{x}) < 0 \text{ for all } i\}$.

Properties.

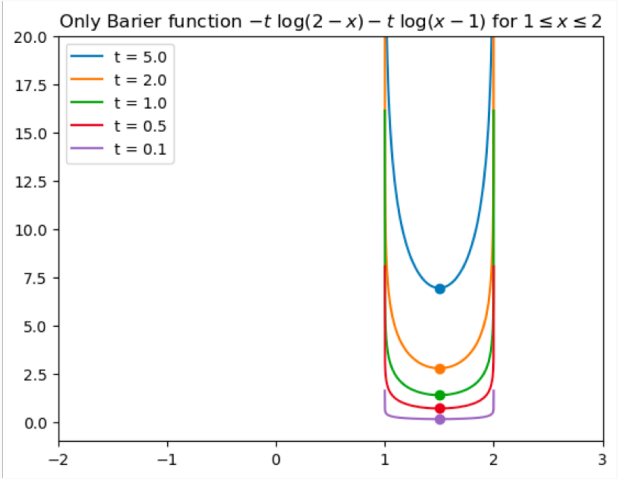
- Φ is smooth on the strict interior.
- $\Phi(\mathbf{x}) \to +\infty$ as $\mathbf{x}$ approaches the boundary (any $g_i(\mathbf{x}) \to 0^-$).
- If each g_i is convex, Φ is convex.

Gradient (use chain rule, $\frac{d}{dz}\log z = 1/z$):

$$\nabla\Phi(\mathbf{x}) = \sum_{i=1}^{m} \frac{\nabla g_i(\mathbf{x})}{-g_i(\mathbf{x})}.$$

The quantity $\dfrac{1}{-g_i(\mathbf{x})}$ will become the "barrier estimate" of the dual multiplier s_i. Hold that thought.

The log barrier alone: a bathtub on $[1, 2]$

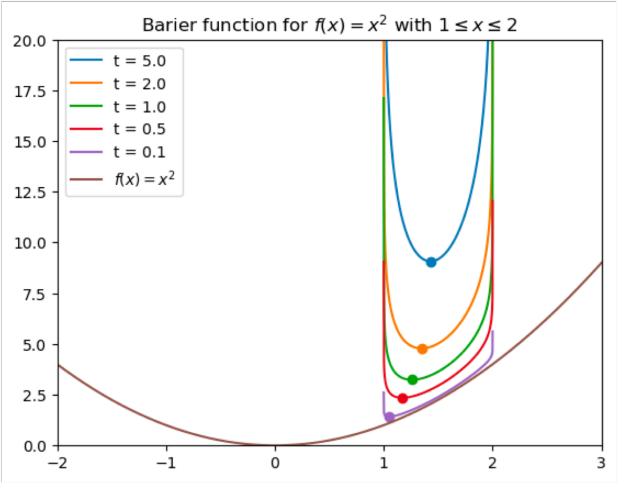


$$\Phi_t(x) = -t\log(2-x) - t\log(x-1)$$

On the interval $[1, 2]$:

- $+\infty$ at both endpoints.
- Smooth, strictly convex inside.
- Its unique minimizer is the **analytic center** $x = 1.5$.
- Shrinks uniformly as $t \downarrow 0$, but never lets you touch the boundary.

Barrier + objective: the 1-D central path

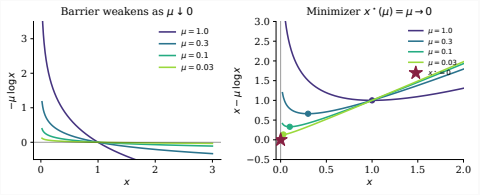


$$\min_x \; x^2 + t\,[-\log(2-x) - \log(x-1)]$$

What to see:

- $t = 5$: barrier dominates, minimizer near analytic center.
- $t \downarrow 0$: the bowl flattens; minimizer slides toward the unconstrained optimum of x^2 (at $x = 1$, the active boundary).
- Dots trace the **central path** $x^\star(t)$ as t sweeps.

Why each $\mu > 0$ is a nice unconstrained problem



For the toy LP $\min x$ s.t. $x \geq 0$:

$$\min_x \; x - \mu \log x \;\Rightarrow\; x^\star(\mu) = \mu.$$

Dual variable: $s(\mu) = \mu / x^\star(\mu) = 1$. Complementarity $sx = \mu$ — **not zero, but controlled**.

Two ideas in one slide:

- Smooth *unconstrained* problem per μ.
- Complementarity $s \cdot x = \mu$ — tunable, and $\to 0$ as $\mu \downarrow 0$.

The barrier problem

For **barrier parameter** $\mu > 0$, define

$$B_\mu(\mathbf{x}) \;:=\; f(\mathbf{x}) \;-\; \mu \sum_{i=1}^{m} \log(-g_i(\mathbf{x}))$$

and solve the *barrier problem*

$$(\mathrm{P}_\mu) \qquad \min_{\mathbf{x}} \; B_\mu(\mathbf{x}) \quad \text{s.t.} \quad h_j(\mathbf{x}) = 0 \;\; (j = 1, \ldots, p).$$

- Equalities remain; only inequalities are folded into the objective.
- (P_μ) is smooth, with only equality constraints — solvable by Newton's method on the Lagrangian, which you already know.
- Solve (P_μ) for $\mu_1 > \mu_2 > \cdots \to 0$, warm-starting each from the previous solution.

This is called a *sequential unconstrained minimization technique* (SUMT), originated by Fiacco & McCormick (1968). Karmarkar's 1984 LP breakthrough gave it polynomial-time status.

Roadmap

Central path: what the KKT of (P_μ) looks like

Stationarity of (P_μ) (no inequalities, just equality constraints):

$$\nabla f(\mathbf{x}) + \mu \nabla \Phi(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) = 0.$$

Expand the barrier gradient:

$$\nabla f(\mathbf{x}) + \sum_i \underbrace{\frac{\mu}{-g_i(\mathbf{x})}}_{=: s_i(\mu)} \nabla g_i(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) = 0.$$

Three observations:

- $s_i(\mu) > 0$ (since $g_i < 0$ at interior points).
- The stationarity equation *matches the KKT stationarity condition exactly* — with s playing the role of the inequality multiplier.
- Complementarity becomes $s_i(\mu) \cdot (-g_i(\mathbf{x})) = \mu$, i.e. **perturbed complementarity**

$$\boxed{s_i \cdot (-g_i(\mathbf{x})) = \mu}$$

The perturbed KKT system

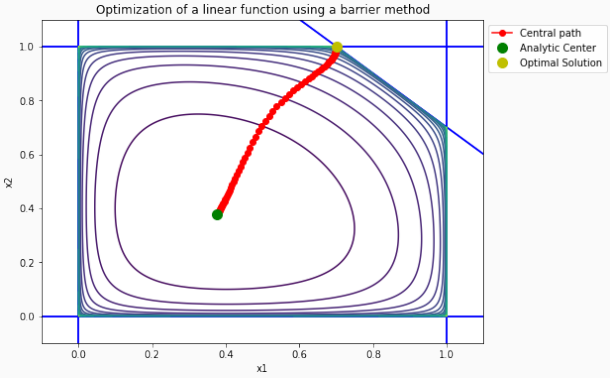
The **central path** is the curve $\{\mathbf{x}(\mu), \mathbf{y}(\mu), \mathbf{s}(\mu) : \mu > 0\}$ satisfying

$$\begin{aligned}
\nabla f(\mathbf{x}) + \sum_i s_i \nabla g_i(\mathbf{x}) + \sum_j y_j \nabla h_j(\mathbf{x}) &= 0 && \text{(stationarity)} \\
h_j(\mathbf{x}) &= 0 && \text{(eq. feas.)} \\
g_i(\mathbf{x}) &< 0 && \text{(ineq. strict feas.)} \\
s_i &> 0 && \text{(dual strict feas.)} \\
s_i \cdot (-g_i(\mathbf{x})) &= \mu && \text{(perturbed complementarity)}
\end{aligned}$$

Compare with standard KKT: the only change is $s_i g_i = 0$ becomes $s_i \cdot (-g_i) = \mu$. **As $\mu \to 0$, the central path $\to$ a KKT point.**

The central path exists and is unique (under standard convexity + Slater) — we'll take this as given. See Nocedal–Wright Thm. 19.2, Boyd–Vandenberghe §11.2.

2D picture: the central path for an LP



$$\min\ c^\top \mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} \le b$$

The contours are level sets of the log-barrier Φ. As μ sweeps $\infty \to 0$:

- Large μ: **analytic center** (green).
- Small μ: **optimal vertex** (yellow).
- In between: smooth curve in the strict interior.

Key fact. Each $\mathbf{x}^\star(\mu)$ is a *unique* minimizer — one well-conditioned Newton problem, no active-set combinatorics.

Worked 2D example: computing the central path explicitly

Problem. $\min -x_1$ s.t. $x_1 + x_2 \le 1,\ x_1, x_2 \ge 0$. Optimum $\mathbf{x}^\star = (1, 0)$, $f^\star = -1$.

Barrier problem (slack $s = 1 - x_1 - x_2$):

$$(\mathrm{P}_\mu) \quad \min_{x_1, x_2, s > 0} \; -x_1 - \mu\big[\log s + \log x_1 + \log x_2\big].$$

Stationarity ($\partial s / \partial x_i = -1$):

$$\partial_{x_1}: \; -1 + \mu\big(\tfrac{1}{s} - \tfrac{1}{x_1}\big) = 0, \qquad \partial_{x_2}: \; \mu\big(\tfrac{1}{s} - \tfrac{1}{x_2}\big) = 0 \Rightarrow s = x_2.$$

From $s = x_2$: $1 - x_1 - x_2 = x_2 \Rightarrow x_2(\mu) = s(\mu) = \frac{1 - x_1}{2}$.

Substitute into first equation: $\mu(3x_1 - 1) = x_1(1 - x_1)$, so

$$x_1^2 + (3\mu - 1)x_1 - \mu = 0 \quad \Rightarrow \quad \boxed{x_1(\mu) = \tfrac{1 - 3\mu + \sqrt{9\mu^2 - 2\mu + 1}}{2}, \;\; x_2(\mu) = \tfrac{1 - x_1}{2}.}$$

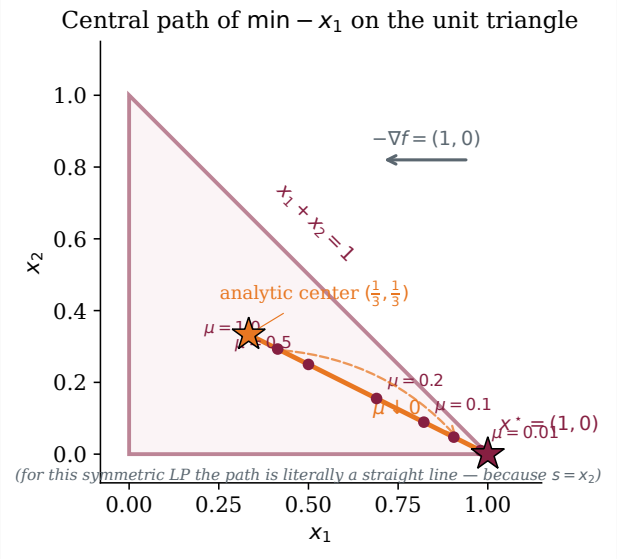
Take $\mu \to 0$.

μ	1	10^{-1}	10^{-2}	10^{-3}	$\to 0$
x_1	0.414	0.822	0.980	0.998	$\to 1$
x_2	0.293	0.089	0.0099	0.00100	$\to 0$
f	-0.414	-0.822	-0.980	-0.998	$\to -1$

At $\mu = 0$: $\sqrt{1} = 1 \Rightarrow x_1(0) = 1$, $x_2(0) = 0$. **LP optimum recovered exactly.**✓

For small μ: $x_1 \approx 1 - \mu$, $x_2 \approx \mu/2$. Duality gap $\approx m\mu$ where $m = 3$ is the number of inequality constraints (two bounds + one linear).

Picture: the 2D central path as $\mu \to 0$



Reading the figure.

- Triangle: feasible region $\{x_1 + x_2 \le 1,\ x_1, x_2 \ge 0\}$.
- Orange curve: central path $\{(x_1(\mu), x_2(\mu))\}$.
- Large μ: path starts at the **analytic center** $(\frac{1}{3}, \frac{1}{3})$.
- $\mu \downarrow 0$: path runs to the **optimal vertex** $(1, 0)$, following $-\nabla f = (1, 0)$.

Why a line, not a curve? For this specific LP the symmetry $s = x_2$ forces $x_2 = (1 - x_1)/2$ for every μ. Most LPs have genuinely curved central paths.

Every LP IPM does exactly this. Choose a warm μ_0, solve the Newton system to stay near the path, decrease μ, repeat. No active-set combinatorics.

Roadmap

What could go wrong with Newton on a barrier?

Newton's method converges quadratically *in a neighborhood* of the solution. Two things we need to nail down:

1. **Size of that neighborhood.** Is it 10^{-3}? 10^{-10}? Does it shrink as $\mu \to 0$?
2. **Step length we can take.** Full steps can overshoot the barrier's domain; we need a principled damping rule.

The classical story (Boyd–Vandenberghe, Ch. 9): if the Hessian $\nabla^2 F$ is Lipschitz and $\nabla^2 F \succeq mI$, then Newton converges quadratically in a ball of size $\sim m/L$. But L and m depend on coordinates, units, problem scaling — not intrinsic.

Nesterov's fix (1988). Replace "Lipschitz Hessian" with a *self-referential* inequality: the third derivative of F is bounded by the second derivative *to the* $3/2$ *power*. The resulting bounds are **affine-invariant** — independent of coordinates.

Self-concordance: the definition

A convex C^3 function $F : \mathrm{int}(K) \to \mathbb{R}$ is **self-concordant** if

$$\boxed{|F'''(\mathbf{x})[\mathbf{v},\mathbf{v},\mathbf{v}]| \;\le\; 2\left(F''(\mathbf{x})[\mathbf{v},\mathbf{v}]\right)^{3/2}} \qquad \forall\, \mathbf{x} \in \mathrm{int}(K),\ \forall\, \mathbf{v}.$$

Why the $3/2$? Dimensional analysis. Rescale the direction $\mathbf{v} \to \lambda\mathbf{v}$ — each bracket slot picks up a factor λ:

$$F''[\mathbf{v},\mathbf{v}] \to \lambda^2 F''[\mathbf{v},\mathbf{v}], \qquad F'''[\mathbf{v},\mathbf{v},\mathbf{v}] \to \lambda^3 F'''[\mathbf{v},\mathbf{v},\mathbf{v}].$$

For $|F'''| \le c\,(F'')^p$ to be preserved under rescaling, need $\lambda^3 = (\lambda^2)^p$, so $p = \frac{3}{2}$.

Any other exponent would depend on your choice of units — strong for small λ, weak for large (or vice versa). The $3/2$ makes the definition **invariant under affine reparameterization** $\mathbf{x} \mapsto A\mathbf{x} + \mathbf{b}$: rescale or rotate coordinates, and SC still holds with the same constant.

One-dimensional examples:

- $F(x) = -\log x$: $F'' = 1/x^2$, $F''' = -2/x^3$. $2/x^3 = 2(1/x^2)^{3/2}$ — **equality**.✓
- $F(x) = x^2$: $F''' = 0 \le 2 \cdot 2^{3/2}$.✓ (every quadratic is SC)
- $F(x) = x^4$: fails near $x = 0$ ($24|x|$ vs. $2(12x^2)^{3/2}$). **Not SC.**

Unpacking the brackets: restriction to a ray

The bracket notation is a trick for *compressing directional derivatives*. The cleanest way to read it:

Restrict F to a ray. Fix $\mathbf{x}, \mathbf{v}$ and define the 1-variable function

$$\phi(t) \;:=\; F(\mathbf{x} + t\mathbf{v}) \qquad (\phi : \mathbb{R} \to \mathbb{R}).$$

Then the brackets are just the t-derivatives of ϕ evaluated at $t = 0$:

$$F'(\mathbf{x})[\mathbf{v}] = \phi'(0), \qquad F''(\mathbf{x})[\mathbf{v}, \mathbf{v}] = \phi''(0), \qquad F'''(\mathbf{x})[\mathbf{v}, \mathbf{v}, \mathbf{v}] = \phi'''(0).$$

Why. By the chain rule, $\phi'(t) = \nabla F(\mathbf{x} + t\mathbf{v}) \cdot \mathbf{v}$. Differentiating again, $\phi''(t) = \mathbf{v}^\top \nabla^2 F(\mathbf{x} + t\mathbf{v})\, \mathbf{v}$. Each derivative in t pulls out one more factor of $\mathbf{v}$.

So the SC inequality $|F'''[\mathbf{v}, \mathbf{v}, \mathbf{v}]| \le 2(F''[\mathbf{v}, \mathbf{v}])^{3/2}$ says:

$$|\phi'''(0)| \le 2\,\phi''(0)^{3/2} \qquad \text{for every } \mathbf{x} \text{ and every direction } \mathbf{v}.$$

A 1D condition that must hold along every ray out of every point.

Punch line. F is self-concordant iff its *1D restriction to every line* satisfies the same inequality. Affine invariance is automatic.

The 3-tensor F''' and what $[\mathbf{v}, \mathbf{v}, \mathbf{v}]$ does to it

Hierarchy of derivative objects:

	Object	Components
$F'(\mathbf{x})$	vector	$[\partial F/\partial x_i]$
$F''(\mathbf{x})$	matrix (Hessian)	$[\partial^2 F/\partial x_i \partial x_j]$
$F'''(\mathbf{x})$	**3-tensor**	$T_{ijk} := \partial^3 F/\partial x_i \partial x_j \partial x_k$

A 3-tensor is just a 3D array — think of a Hessian with one extra index, an $n \times n \times n$ box of numbers.

Contracting against $\mathbf{v}$. For the Hessian, a bilinear form:

$$F''(\mathbf{x})[\mathbf{v}, \mathbf{v}] = \sum_{i,j} H_{ij}\, v_i\, v_j = \mathbf{v}^\top H \mathbf{v}.$$

For the 3-tensor, contract all three slots with $\mathbf{v}$ — a scalar cubic in $\mathbf{v}$:

$$\boxed{F'''(\mathbf{x})[\mathbf{v}, \mathbf{v}, \mathbf{v}] = \sum_{i,j,k} T_{ijk}\, v_i\, v_j\, v_k.}$$

Separable example. $F(x_1, x_2) = -\log x_1 - \log x_2$ has all mixed partials zero; only $T_{111} = -2/x_1^3$ and $T_{222} = -2/x_2^3$ survive. Triple sum collapses to

$$F'''[\mathbf{v}, \mathbf{v}, \mathbf{v}] = -\tfrac{2v_1^3}{x_1^3} - \tfrac{2v_2^3}{x_2^3}.$$

Cost. Never materialize the full n^3 tensor; we only need $T[\mathbf{v}, \mathbf{v}, \mathbf{v}]$. $O(n^3)$ worst case, $O(n)$ when sparse.

The SC inequality in 1D and 2D: worked

1D: $F(x) = -\log x$ **on** $(0, \infty)$**.**

$$F''(x) = 1/x^2, \qquad F'''(x) = -2/x^3.$$

Checking SC: $|F'''| = 2/x^3$ and $2(F'')^{3/2} = 2 \cdot (1/x^2)^{3/2} = 2/x^3$. **Equality** holds everywhere.

$\Rightarrow -\log x$ is *exactly* self-concordant. The log is the canonical 1D SC function, and the constant 2 in the definition was chosen to make this identity work.

2D: $F(x_1, x_2) = -\log x_1 - \log x_2$ **on** $\mathbb{R}^2_+$**.** (The LP barrier in 2D.)

Hessian is diagonal:

$$\nabla^2 F(\mathbf{x}) = \begin{pmatrix} 1/x_1^2 & 0 \\ 0 & 1/x_2^2 \end{pmatrix}, \qquad F''[\mathbf{v}, \mathbf{v}] = \frac{v_1^2}{x_1^2} + \frac{v_2^2}{x_2^2}.$$

3-tensor — only diagonal terms survive (the function is separable):

$$\frac{\partial^3 F}{\partial x_1^3} = -\frac{2}{x_1^3}, \quad \frac{\partial^3 F}{\partial x_2^3} = -\frac{2}{x_2^3}, \quad \text{all mixed partials} = 0.$$

$$F'''[\mathbf{v}, \mathbf{v}, \mathbf{v}] = -\frac{2v_1^3}{x_1^3} - \frac{2v_2^3}{x_2^3}.$$

Check SC. Let $u_i = v_i / x_i$. Then

$$F''[\mathbf{v}, \mathbf{v}] = u_1^2 + u_2^2, \qquad |F'''[\mathbf{v}, \mathbf{v}, \mathbf{v}]| = 2|u_1^3 + u_2^3| \le 2(u_1^2 + u_2^2)^{3/2}.$$

The last inequality is a standard p-norm bound: $\|\mathbf{u}\|_3 \le \|\mathbf{u}\|_2$ in $\mathbb{R}^2$. ✓

Moral. Self-concordance of sums: if each F_i is SC in its own variable, $\sum F_i$ is SC in the product. The LP barrier $-\sum \log x_i$ inherits SC from the 1D log, for any n.

Self-concordant barrier: one more inequality

A self-concordant function F is a **self-concordant barrier** for K if additionally there exists $\nu \geq 1$ such that

$$\boxed{|F'(\mathbf{x})[\mathbf{v}]|^2 \;\leq\; \nu \cdot F''(\mathbf{x})[\mathbf{v},\mathbf{v}]} \qquad \forall\, \mathbf{x} \in \operatorname{int}(K),\ \forall\, \mathbf{v}.$$

The smallest such ν is the **barrier parameter** (or **complexity parameter**) of F.

Equivalent formulation. Writing $\mathbf{v}^* = -(F''(\mathbf{x}))^{-1}F'(\mathbf{x})$ (the Newton direction for the barrier), the squared Newton decrement

$$\lambda_F(\mathbf{x})^2 \;=\; F'(\mathbf{x})^\top (F''(\mathbf{x}))^{-1} F'(\mathbf{x}) \;\leq\; \nu.$$

Reading it. ν upper-bounds how "big" the gradient of F can be relative to its Hessian. Small ν = the barrier is "tight" in a sense we'll make quantitative.

The barrier parameter drives complexity. We'll see that the number of IPM iterations to reduce μ by a factor $1/\varepsilon$ is

$$K \;=\; O\big(\sqrt{\nu}\,\log(1/\varepsilon)\big).$$

Smaller $\nu \Rightarrow$ fewer iterations.

The Newton decrement is the convergence rate

For a self-concordant F, define the **Newton decrement** at $\mathbf{x}$:

$$\lambda_F(\mathbf{x}) \;=\; \sqrt{F'(\mathbf{x})^\top (F''(\mathbf{x}))^{-1} F'(\mathbf{x})},$$

the length of the Newton step $\Delta\mathbf{x} = -(F'')^{-1}F'$ in the Hessian-induced norm.

Suboptimality certificate. For $\lambda_F(\mathbf{x}) \le 0.68$:

$$F(\mathbf{x}) - F(\mathbf{x}^\star) \;\le\; \lambda_F(\mathbf{x})^2.$$

$\lambda_F \le 10^{-4}$ gives optimality gap $\le 10^{-8}$ — so λ_F *is* the stopping criterion.

Convergence rate (full Newton step):

- **Quadratic phase** ($\lambda_F < 0.38$): $\lambda_F(\mathbf{x}^+) \le \big(\lambda_F/(1-\lambda_F)\big)^2$ — classical Newton quadratic convergence.
- **Damped phase** ($\lambda_F \ge 0.38$): step $\alpha = 1/(1+\lambda_F)$, F drops by at least $\gamma \ge 0.09$ per iteration.

Iteration count:

$$K \;\le\; \underbrace{(F(\mathbf{x}^0) - F(\mathbf{x}^\star))/\gamma}_{\text{damped phase (finite)}} + \underbrace{\log_2\log_2(1/\varepsilon)}_{\text{quadratic phase}}.$$

The log-log term is why going from $\varepsilon = 10^{-4}$ to 10^{-16} costs only two more Newton steps.

In path-following, the short-step schedule keeps iterates in the quadratic region ⇒ **one** Newton step per μ update. That's why the master bound is $O(\sqrt{\nu}\log(1/\varepsilon))$ outer iterations, each nearly free inside.

The master complexity theorem

Theorem [Nesterov–Nemirovski 1994]. Let K be a solid convex cone with self-concordant barrier F of parameter ν. The path-following IPM

$$\mathbf{x}^\star(\mu) = \arg\min_{\mathbf{x}\in\operatorname{int}(K)} \mathbf{c}^\top \mathbf{x} + \mu F(\mathbf{x}), \qquad \mu \downarrow 0$$

converges to an ε-approximate solution in

$$\boxed{K = O(\sqrt{\nu}\,\log(\nu/\varepsilon))} \text{ iterations.}$$

Total work = (outer iterations) $\times$ (cost of one Newton system) = $O(\sqrt{\nu}\log(1/\varepsilon)) \times O(\text{sparse linear solve})$.

Consequence. Because *every* convex cone admits a self-concordant barrier (universal barrier theorem), every convex problem is solvable in polynomial time — a radical generalization of Karmarkar's 1984 LP result.

Why this matters for modeling. Smaller $\nu \Rightarrow$ fewer outer iterations. SOCP has $\nu = 2$ per cone while the orthant has $\nu = n$: an SOC reformulation of a large LP *can run faster in iteration count* than the LP itself.

Roadmap

The cone zoo: five cones you'll meet in CVXPY

Every modern convex solver works on a Cartesian product of these cones.

Cone	Definition	ν
Zero cone $\{0\}^m$	— for equality constraints	0
Nonneg orthant $\mathbb{R}^n_+$	$\{x : x_i \geq 0\}$	n
Second-order cone $\mathcal{Q}^n$	$\{(t, \mathbf{x}) : t \geq \|\mathbf{x}\|_2\}$	2
PSD cone $\mathbb{S}^n_+$	$\{X = X^\top : X \succeq 0\}$	n
Exponential cone K_{exp}	$\text{cl}\{(x, y, z) : ye^{x/y} \leq z,\ y > 0\}$	3
Power cone K_α	$\{(x, y, z) : \|z\| \leq x^\alpha y^{1-\alpha},\ x, y \geq 0\}$	3

Composition rule. Cartesian products of SC barriers are SC barriers with ν's summing:

$$F_{K_1 \times K_2}(\mathbf{x}_1, \mathbf{x}_2) = F_1(\mathbf{x}_1) + F_2(\mathbf{x}_2), \qquad \nu_{K_1 \times K_2} = \nu_1 + \nu_2.$$

So a problem mixing 10 SOC constraints and a 50×50 PSD gets $\nu = 10 \cdot 2 + 50 = 70$.

Solver support today (from CVXPY's perspective):

Barrier #1: the nonnegative orthant

Cone. $\mathbb{R}^n_+ = \{\mathbf{x} \in \mathbb{R}^n : x_i \geq 0 \ \forall i\}$.

Barrier. $F(\mathbf{x}) = -\sum_{i=1}^n \log x_i$.

Derivatives. $\nabla F(\mathbf{x}) = (-1/x_i)_i$, $\nabla^2 F(\mathbf{x}) = \operatorname{diag}(1/x_i^2)$, diagonal.

Verify self-concordance. Separates into 1D: for each i, $f_i(x) = -\log x$. We computed $|f'''| = 2/x^3 = 2(f'')^{3/2}$. ✓

Verify the ν-bound. The Newton decrement squared is

$$\lambda_F(\mathbf{x})^2 = \sum_{i=1}^n \frac{(1/x_i)^2}{1/x_i^2} = \sum_{i=1}^n 1 = n.$$

So $\nu = n$ is tight. (Maximized when $\mathbf{v}$ points in direction $(x_1, \ldots, x_n)$.)

Consequence. LP with n non-negativity constraints: $K = O(\sqrt{n}\log(1/\varepsilon))$. This *is* the $O(\sqrt{n})$ Renegar-Gonzaga path-following bound.

This barrier is what IPOPT uses internally on the slack variables $\mathbf{s}^g$ (after it reformulates $g_i(\mathbf{x}) \leq 0$ as $g_i(\mathbf{x}) + s_i^g = 0, s_i^g \geq 0$).

Barrier #2: the second-order cone

Cone. $\mathcal{Q}^n = \{(t, \mathbf{x}) \in \mathbb{R} \times \mathbb{R}^{n-1} : t \geq \|\mathbf{x}\|_2\}$. An ice-cream cone.

Barrier. $F(t, \mathbf{x}) = -\log(t^2 - \|\mathbf{x}\|_2^2)$.

Let $\phi(t, \mathbf{x}) = t^2 - \|\mathbf{x}\|^2$ (concave on $\mathcal{Q}^n$). $F = -\log \phi$.

Gradient. $\nabla F = -(1/\phi)\,\nabla\phi = -\frac{1}{\phi}\begin{pmatrix} 2t \\ -2\mathbf{x} \end{pmatrix}$.

Hessian (after computation): $\nabla^2 F = \frac{4}{\phi}\begin{pmatrix} 1 & 0 \\ 0 & -I \end{pmatrix} + \frac{1}{\phi^2}(\nabla\phi)(\nabla\phi)^\top$. Not diagonal. But $\succ 0$ on $\operatorname{int}(\mathcal{Q}^n)$.

Verify $\nu = 2$. After a short calculation (details: Nesterov–Todd 1997),

$$\lambda_F(t, \mathbf{x})^2 = \nabla F^\top (\nabla^2 F)^{-1} \nabla F \leq 2.$$

Why this matters: dimension doesn't affect complexity. Regardless of how big n is, one SOC contributes only $\nu = 2$ to the master theorem. So SOCP with k cones of *any* size: $\nu_{\text{total}} = 2k$, and $K = O(\sqrt{k}\log(1/\varepsilon))$ outer iterations.

Practical consequence. Reformulating a big LP as an SOCP (say, by representing the cost $\|\mathbf{c}\|_1$ with an SOC) can *reduce* iteration count. Rule of thumb: pick the cone that matches your constraint geometry, not the one that's linear.

Barrier #3: the positive semidefinite cone

Cone. $\mathbb{S}^n_+ = \{X \in \mathbb{R}^{n\times n} : X = X^\top,\ X \succeq 0\}$ — dimension $n(n+1)/2$.

Barrier. $F(X) = -\log\det X$.

Why it works. $\log\det$ is concave on $\mathbb{S}^n_+$ (Minkowski's inequality). $\det X \to 0$ as X hits the boundary (an eigenvalue $\to 0$), so $F \to +\infty$.

Derivatives (matrix calculus):

$$\nabla F(X) = -X^{-1}, \qquad \nabla^2 F(X)[H,H] = \operatorname{tr}(X^{-1}HX^{-1}H).$$

Newton decrement. $\lambda_F(X)^2 = \operatorname{tr}(X^{-1}X^{-1}) = \ldots = n$ (for the gradient direction). So $\nu = n$.

Eigenvalue intuition. If $X = Q\Lambda Q^\top$ with $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$:

$$F(X) = -\sum_{i=1}^{n} \log \lambda_i.$$

The SDP barrier is the LP barrier on eigenvalues. Same $\nu = n$, same $O(\sqrt{n})$ complexity per PSD block.

A 50×50 SDP contributes $\nu = 50$. Its dimension (variables) is $50 \cdot 51/2 = 1275$. IPM iterations scale with $\sqrt{50} \approx 7$, not $\sqrt{1275} \approx 36$ — SDPs converge remarkably fast in *iteration count*, though each iteration can be expensive.

Barriers #4 and #5: exp and power cones

Exponential cone.

$$K_{\exp} = \mathrm{cl}\{(x, y, z) \in \mathbb{R}^3 : y \exp(x/y) \le z,\ y > 0\}.$$

Captures constraints involving log, exp, entropy, KL-divergence.

Barrier. Not as clean as the others. Serrano (2015) gave the explicit form:

$$F(x, y, z) = -\log\big(y \log(z/y) - x\big) - \log z - \log y, \qquad \nu = 3.$$

Power cone.

$$K_\alpha = \{(x, y, z) \in \mathbb{R}^3 : |z| \le x^\alpha y^{1-\alpha},\ x, y \ge 0\}, \quad \alpha \in (0, 1).$$

Captures p-norm constraints and x^p for rational p.

Barrier.

$$F(x, y, z) = -\log(x^{2\alpha} y^{2(1-\alpha)} - z^2) - \log x - \log y, \qquad \nu = 3.$$

Why these cones matter in CVXPY. Many "awkward" convex constraints — entropy, log-sum-exp, p-norms, geometric means — are *not* expressible in LP + SOC + SDP. Exp and power cones make them tractable. Solver ECOS handles LP + SOC + exp; CLARABEL adds power + SDP.

Where does a new barrier come from?

Design recipe (most tractable SC barriers follow it):

Step 1. Express the cone as $K = \{\mathbf{x} : \phi_i(\mathbf{x}) \geq 0\}$ for concave ϕ_i.

Step 2. Try $F = -\sum_i \log \phi_i$.

Step 3. Verify self-concordance via the inequality (often reduces to known identities for sum/composition/affine-restriction).

Worked examples:

- $\mathbb{R}^n_+$: $\phi_i(\mathbf{x}) = x_i$ (affine, hence concave). $F = -\sum \log x_i$. $\nu = n$.
- $\mathcal{Q}^n$: $\phi(t, \mathbf{x}) = t^2 - \|\mathbf{x}\|^2$ (concave). $F = -\log \phi$. $\nu = 2$.
- $\mathbb{S}^n_+$: $\phi(X) = \det X$ (log-concave, not concave — so use $-\log\det$ directly).

Universal existence. Nesterov–Nemirovski (1994, Thm. 2.5.1): *every* n-dimensional convex cone admits an SC barrier with $\nu = O(n)$. Their construction uses

$$F_{\text{univ}}(\mathbf{x}) = \log \operatorname{vol}\{\mathbf{y} \in K^* : \langle \mathbf{y}, \mathbf{x} \rangle \leq 1\}.$$

This is a **theoretical existence result** — not computable in practice. All cones you'll meet in CVXPY have hand-designed, tractable barriers.

Roadmap

Coding Break #1 — what to pull up

Notebook: `part1_solvers/02_cvxpy_dcp.ipynb`

Why here. You've seen what a log-barrier IPM does on the 2D LP by hand, and you've seen the complexity theorem that makes it polynomial for every cone. Now let's watch CVXPY dispatch a conic IPM on the same kind of problem — no derivatives by hand, one `prob.solve()`.

What we'll run:

1. **2D LP sanity check** — the same problem we just solved. Verify $(1, 0)$ and inspect `prob.solver_stats`.
2. **LASSO** — $\min \frac{1}{2}\|A\mathbf{x}-\mathbf{b}\|^2 + \lambda\|\mathbf{x}\|_1$. An SOC + LP problem.
3. **Markowitz portfolio** — a quadratic objective with simplex constraint.
4. **Dual variables** — `constraint.dual_value`. The KKT multipliers we derived, one line of code.
5. **Swap solvers** — ECOS vs. CLARABEL vs. SCS on the same problem. Compare

CVXPY basics: a two-minute refresher

```python
import cvxpy as cp
import numpy as np

n = 10
A = np.random.randn(20, n)
b = np.random.randn(20)
lam = 0.1

x = cp.Variable(n)
obj = cp.Minimize(0.5 * cp.sum_squares(A @ x - b) + lam * cp.norm1(x))
prob = cp.Problem(obj)
prob.solve()

print("x*:", x.value)
print("solver:", prob.solver_stats.solver_name)
print("iters:", prob.solver_stats.num_iters)
print("solve time:", prob.solver_stats.solve_time)
```

What happened internally:

1. `cp.sum_squares`: squared ℓ_2 norm — DCP attributes (convex, nonneg, nondecreasing in $|\cdot|$).
2. `cp.norm1`: ℓ_1 norm — convex, nonneg, nondecreasing in $|\cdot|$.
3. Composition rule passes $\Rightarrow$ problem is DCP-compliant.
4. `prob.solve()`: graph-implementation rewrite $\rightarrow$ standard conic form $\rightarrow$ ECOS/CLARABEL.

Dual recovery: if you add `constraints = [x >= 0]` and call `constraints[0].dual_value` post-solve, you get the Lagrange multiplier vector.

Roadmap

Why DCP? Convexity is undecidable in general

Problem. You write

$$f(\mathbf{x}) = \log\bigl(\|\mathbf{x}\|^2 + \exp(x_1) - \cos(x_2)\bigr).$$

Is f convex?

Undecidability. Deciding whether an arbitrary expression is convex is undecidable in general (Ahmadi 2009, for polynomials of degree ≥ 4). No algorithm can correctly classify every convex expression.

Consequence. CVXPY cannot just "check convexity". It would either reject convex problems (conservative) or accept non-convex ones (unsafe).

DCP's way out. Don't decide convexity. *Construct* convexity.

- Restrict the grammar to a set of *atomic functions* with known curvature.
- Combine them only with rules that *preserve* convexity.
- A problem that parses in this restricted grammar is convex *by construction*.

Tradeoff. You give up some convex problems (e.g. $\sqrt{x^2+1}$ is convex but not DCP-expressible as written). You gain: a *proof* of convexity that comes for free with every CVXPY program that parses.

DCP attributes: every atom carries three labels

Every function in CVXPY's atom library has three **curvature attributes**.

Curvature:

- **constant** — doesn't depend on variables.
- **affine** — $ax + b$; both convex and concave.
- **convex** — $f(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y})$.
- **concave** — the opposite.

Sign: positive, **negative**, **zero**, **unknown**.

Monotonicity (per argument): nondecreasing, **nonincreasing**, **nonmonotonic**.

Atom	Curvature	Sign	Monotonicity
`square(x)`	convex	positive	nonmonotonic
`abs(x)`	convex	positive	nonmonotonic
`exp(x)`	convex	positive	nondecreasing
`log(x)`	concave	unknown	nondecreasing
`norm2(x)`	convex	positive	nondecreasing in $\lvert x \rvert$
`log_sum_exp(x)`	convex	unknown	nondecreasing
`sqrt(x)`	concave	positive	nondecreasing

The composition rule

For $f(\mathbf{x}) = h(g_1(\mathbf{x}), g_2(\mathbf{x}), \ldots, g_k(\mathbf{x}))$, DCP concludes f is **convex** if:

> h is convex, *and* for each i, *one of*:
>
> - h is nondecreasing in arg i *and* g_i is convex,
> - h is nonincreasing in arg i *and* g_i is concave,
> - g_i is affine.

Concave case: swap "convex" and "concave" throughout.

Works: `log_sum_exp(A @ x + b)`

- `log_sum_exp` is convex, nondecreasing in each arg.
- `A @ x + b` is affine $\Rightarrow$ rule passes.

$\Rightarrow$ convex. ✓

Fails: `log(exp(x) - exp(y))`

- `log` is concave, nondecreasing. But we're composing with `exp(x) - exp(y)` — convex minus convex, curvature *unknown*.
- Concave rule needs concave inside for nondecreasing args; unknown fails.

$\Rightarrow$ "DCP violation: does not follow DCP rules".

"Fails DCP" $\neq$ "is non-convex." CVXPY is conservative: some convex expressions fail DCP because they can't be *verified* by the composition rule. Sometimes you can rewrite.

DCP-valid, DCP-invalid, and "true but not DCP"

DCP-valid (clearly convex, clearly parses):

- `cp.square(x - 3)`: square convex nonmonotonic; `x - 3` affine.
- `cp.norm(A @ x - b, 2)`: SOC epigraph.
- `cp.log_sum_exp(W @ x)`: softmax log-partition function.

DCP-invalid (genuinely non-convex):

- `cp.square(x) - cp.square(y)`: difference of convex. Non-convex in general. Rejected.
- `x * y`: bilinear. Rejected (not convex).

"True but not DCP" (convex, but not as written):

- `cp.sqrt(x**2 + 1)`: convex (composition of convex nondecreasing `sqrt` with convex `x**2+1`). But DCP sees `sqrt` as *concave-nondecreasing*, which needs a concave argument. Composition rule fails.
- *Fix:* rewrite as `cp.norm(cp.vstack([x, 1]), 2)` — an SOC epigraph. Mathematically equivalent, DCP-valid.

DCP is a **proof system**, not a classifier. A rejection doesn't mean non-convex — it means CVXPY can't *verify* convexity via the rule. Half the art of CVXPY is knowing when and how to rewrite.

Roadmap

Coding Break #2 — DCP as a proof system

Notebook: `part1_solvers/02_5_dcp_proof_system.ipynb`

Goal. Stop treating DCP as a black box. Every CVXPY expression has introspection hooks; we'll use them to watch the composition rule fire in real time.

What we'll run:

1. **A DCP-compliant expression** — inspect `.curvature`, `.sign`, `.is_dcp()` as the tree gets built bottom-up.
2. **True but not DCP**: $\sqrt{x^2+1}$. Convex — but CVXPY rejects it. See *why* via the attributes, then rewrite using `cp.norm`.
3. **Composition rule at work** — `exp(convex)` passes, `exp(concave)` fails.
4. **Signs matter** — positive vs. negative scalar times a convex expression.
5. **Difference of convex** — genuinely non-convex. Rejected.
6. **Constraints** — `convex` $\leq$ `concave`, `affine` $=$ `affine`, and nothing else.

Roadmap

The graph implementation: each atom knows its epigraph

Key idea. Every DCP atom f comes with a **graph implementation**: a representation of its epigraph

$$\mathrm{epi}(f) = \{(\mathbf{x}, t) : f(\mathbf{x}) \le t\}$$

as a set of cone constraints over fresh auxiliary variables. When CVXPY lifts your problem, each atom is replaced by its graph.

Minimal example: $\|\mathbf{x}\|_2$. The epigraph is

$$\{(\mathbf{x}, t) : \|\mathbf{x}\|_2 \le t\} = \{(\mathbf{x}, t) : (t, \mathbf{x}) \in \mathcal{Q}^{n+1}\}.$$

So `cp.norm(x, 2)` gets replaced by an auxiliary `t` and the constraint `(t, x) in SOC`. The objective (or upper-bounding constraint) then sees `t` instead of `cp.norm(x, 2)`.

ℓ_1 norm. $\|\mathbf{x}\|_1 = \sum_i |x_i|$. Epigraph: $\sum_i u_i \le t,\ -u_i \le x_i \le u_i$. Fresh vars $\mathbf{u}$; LP constraints only.

Max eigenvalue. $\lambda_{\max}(X)$ for symmetric X. Epigraph: $t \ge \lambda_{\max}(X) \iff tI - X \succeq 0$. One LMI.

$-\log\det X$. Requires a sequence of Schur complements plus n exp-cone constraints. Not pretty, but CVXPY handles it for you.

This is called "conic reformulation" or "graph lifting." The lifted problem is equivalent, *is* a cone program in standard form, and can be solved by any conic IPM.

The full CVXPY pipeline, one figure

Input (your Python):

```
x = cp.Variable(n)
obj = cp.Minimize(cp.norm(A @ x - b, 2) + lam * cp.norm1(x))
prob = cp.Problem(obj, [x >= 0])
prob.solve()
```

Pipeline inside `prob.solve()`:

1. **Parse.** Build expression tree. Each node carries curvature + sign + monotonicity.
2. **DCP verification.** Walk the tree bottom-up via the composition rule. If any node's curvature is "unknown", raise `DCPError`.
3. **Graph rewrite.** Replace each atom by its epigraph / hypograph via auxiliary variables. E.g. `cp.norm(A @ x - b, 2)` becomes `t` with `(t, A@x-b) in SOC`.
4. **Canonicalize.** Flatten to standard conic form

$$\min\ \mathbf{c}^\top \mathbf{z} \quad \text{s.t.} \quad A_{\text{eq}}\mathbf{z} = \mathbf{b}_{\text{eq}},\ \mathbf{z} \in K_1 \times K_2 \times \cdots \times K_p.$$

5. **Solve.** Ship to a conic IPM (ECOS, CLARABEL, SCS, MOSEK) that runs the self-concordant-barrier machinery from today.
6. **Unpack.** Map the conic solution back to your original `x.value` and `constraint.dual_value`.

You can see the intermediate form via `prob.get_problem_data(solver="CLARABEL")`. Inspect `c`, `A`, `b`, and the cone list. It's often much bigger than your original — that's the price of auxiliary variables.

Example: what CVXPY does with a norm objective

You write:

$$\min_{\mathbf{x}} \ \|A\mathbf{x} - \mathbf{b}\|_2$$

CVXPY lifts to:

$$\min_{\mathbf{x},t} \ t \quad \text{s.t.} \quad (t, A\mathbf{x} - \mathbf{b}) \in \mathcal{Q}^{m+1}.$$

A cone program with one SOC of size $m+1$. CLARABEL runs an IPM on this.

You write:

$$\min_{\mathbf{x}} \ \|A\mathbf{x} - \mathbf{b}\|_2^2 + \lambda\|\mathbf{x}\|_1$$

CVXPY lifts to:

$$\min_{\mathbf{x},t,\mathbf{u}} \ t^2 + \lambda \mathbf{1}^\top \mathbf{u} \quad \text{s.t.} \quad (t, A\mathbf{x} - \mathbf{b}) \in \mathcal{Q}^{m+1}, \ -\mathbf{u} \le \mathbf{x} \le \mathbf{u}.$$

Note: t^2 is not linear. CVXPY introduces a second auxiliary $s \ge t^2$ and represents it as a *rotated SOC* $(s, t) \in \mathcal{Q}_{\text{rot}}$. Final form:

$$\min_{\mathbf{x},s,\mathbf{u}} \ s + \lambda \mathbf{1}^\top \mathbf{u} \quad \text{s.t.} \quad (s, \tfrac{1}{2}, A\mathbf{x} - \mathbf{b}) \in \mathcal{Q}_{\text{rot}}^{m+2}, \ -\mathbf{u} \le \mathbf{x} \le \mathbf{u}.$$

Conic form: $n + 1 + n = 2n + 1$ variables, one rotated SOC + LP constraints.
ECOS/CLARABEL solves this in 10–20 iterations.

Moral. Your one-line CVXPY problem may blow up into a mid-sized cone program under the hood. This is normal and efficient — the solvers are designed for exactly this structure.

A fancier worked example: elastic-net logistic regression

The DCP-valid expression:

```
beta = cp.Variable(p)
# One atom per data point: log(1 + exp(-y_i * (a_i^T beta)))
loss = cp.sum(cp.logistic(-cp.multiply(y, A @ beta)))   # logistic(z) = log(1+e^z)
reg_l1 = lam1 * cp.norm1(beta)
reg_l2 = 0.5 * lam2 * cp.sum_squares(beta)
prob = cp.Problem(cp.Minimize(loss + reg_l1 + reg_l2))
prob.solve()
```

DCP walk.

- `cp.logistic(z)` atom: convex, nondecreasing.
- `-cp.multiply(y, A @ beta)`: affine (since y is data).
- `cp.sum(convex)`: convex nondecreasing scaling.
- $\lambda_1 \cdot$ `cp.norm1`: convex (positive scaling of convex).
- $\frac{\lambda_2}{2} \cdot$ `cp.sum_squares`: convex (positive scaling of convex).
- Sum of three convex $=$ convex. ✓ **The composition rule passes at every node.**

Now watch CVXPY lift it. Each atom gets replaced by its cone implementation:

Atom	Graph implementation	Cone used
`logistic(z_i)`	$(z_i, 1, e^{z_i} - 1)\ldots$ via 2 exp-cone constraints per point	$K_{\exp}^{2N}$
`norm1(beta)`	$-u_j \le \beta_j \le u_j$; minimize $\mathbf{1}^\top \mathbf{u}$	$\mathbb{R}_+^{2p}$ (LP)
`sum_squares(beta)`	$s \ge \|\beta\|_2^2$ as rotated SOC $(s, 1/2, \beta)$	$\mathcal{Q}_{\text{rot}}^{p+2}$

Logistic regression: the lifted cone program

Introduce auxiliary variables (one per lifted atom):

$$\mathbf{t} \in \mathbb{R}^N, \quad \mathbf{s}_0, \mathbf{s}_1 \in \mathbb{R}^N_+, \quad \mathbf{u} \in \mathbb{R}^p_+, \quad r \in \mathbb{R}_+.$$

Logistic epigraph ($\log(1+e^{z_i}) \le t_i$ is equivalent to $e^{-t_i} + e^{z_i - t_i} \le 1$):

$$(-t_i, 1, s_{0i}) \in K_{\exp},\ (z_i - t_i, 1, s_{1i}) \in K_{\exp},\ s_{0i} + s_{1i} \le 1,\ z_i = -y_i(\mathbf{a}_i^\top \beta).$$

ℓ_1 **epigraph:** $-u_j \le \beta_j \le u_j$ for $j = 1, \dots, p$.

ℓ_2^2 **epigraph:** $(r,\ \tfrac{1}{2},\ \beta) \in \mathcal{Q}^{p+2}_{\mathrm{rot}}$ (rotated SOC, $r \ge \|\beta\|^2$).

Final standard form:

$$\min_{\beta, \mathbf{t}, \mathbf{s}_0, \mathbf{s}_1, \mathbf{u}, r} \ \mathbf{1}^\top \mathbf{t} + \lambda_1 \mathbf{1}^\top \mathbf{u} + \frac{\lambda_2}{2} r \quad \text{s.t.} \quad \begin{array}{c} \text{(linear equalities coupling } z_i, \beta, \mathbf{t}, \mathbf{s}, \mathbf{u}, r) \\ (\beta, \mathbf{t}, \mathbf{s}_0, \mathbf{s}_1, \mathbf{u}, r) \in \underbrace{\mathbb{R}^p}_{\text{free}} \times \underbrace{\mathbb{R}^N}_{\text{free}} \times K_{\exp}^{2N} \times \mathbb{R}^{2p}_+ \times \underbrace{\mathcal{Q}^{p+2}_{\mathrm{rot}}}_{\ell_2} \end{array}$$

Reading the Cartesian product. The solver sees one big cone

$$K_{\mathrm{total}} \ = \ K_{\exp}^{2N} \times \mathbb{R}^{2p}_+ \times \mathcal{Q}^{p+2}_{\mathrm{rot}},$$

with self-concordance parameter (sum of pieces):

$$\nu_{\mathrm{total}} \ = \ 2N \cdot 3 \ + \ 2p \cdot 1 \ + \ 1 \cdot 2 \ = \ 6N + 2p + 2.$$

Master theorem predicts $O(\sqrt{6N + 2p + 2}\, \log(1/\varepsilon))$ outer IPM iterations — polynomial in both data size N and feature count p.

What CVXPY didn't tell you. Your 4-line Python $\rightsquigarrow$ a cone program over a $(p + N + 2N + 2p + 1)$-dimensional variable with $2N$ exp-cones, one rotated SOC, and linear glue. CLARABEL chews this in 15–30 iterations on typical problem sizes.

See the lifting live

Run this to inspect what CVXPY actually built:

```
data, chain, inv = prob.get_problem_data(solver=cp.CLARABEL)

print("n variables:",   data["P"].shape[0])
print("n constraints:", data["A"].shape[0])
print("cone structure:", data["dims"])       # ConeDims object
# Example output for the elastic-net logistic problem (N=50, p=5):
#   n variables: 160
#   n constraints: 360
#   cone structure: (zero: 0, nonneg: 60, exp: 100, soc: [], psd: [], p3d: [])
```

Decoding the cone dims. `zero` = equality constraints; `nonneg` = orthant (LP); `soc` = second-order cones; `exp` = exp cones (counted in triples of rows); `psd` = PSD blocks; `p3d` = power cones.

The lifted problem is typically 3–5× bigger than your original expression — but every constraint is now a cone constraint, and the solver applies its self-concordant-barrier IPM uniformly.

Practical tip. If `prob.solve()` is slow, look at the cone counts. Huge exp counts mean a lot of `log`/`exp` atoms — solvable, but each exp cone contributes $\nu = 3$. Sometimes you can reformulate a `log_sum_exp` as an SOC if you're willing to lose some structure.

Putting it all together: Nesterov–Nemirovski + DCP + conic IPMs

The full chain from your expression to a solution:

1. You write a DCP-compliant optimization problem.
2. DCP verifies convexity *syntactically* via the composition rule.
3. Graph implementations lift each atom to its epigraph cone constraint.
4. CVXPY flattens to standard form: $\min \mathbf{c}^\top \mathbf{z}$ s.t. $A\mathbf{z} = \mathbf{b}$, $\mathbf{z} \in K_1 \times \cdots \times K_p$.
5. Each cone has a self-concordant barrier with known ν.
6. The solver runs path-following IPM with the product barrier, $\nu_{\text{total}} = \sum \nu_i$.
7. Master theorem guarantees $O(\sqrt{\nu_{\text{total}}} \log(1/\varepsilon))$ iterations.

The big picture
Self-concordance is the mathematical foundation that makes CVXPY possible. Without it, the IPM complexity bound exists only for LP. With it, *any* convex cone — and therefore any DCP-compliant convex problem — is solvable in polynomial time.

One more piece of the ecosystem. For problems you solve *repeatedly* with different parameters (like MPC), CVXPY can **pre-compile** the conic reformulation and generate C code via `CVXPYgen`. See notebook `03_cvxpygen_parametric.ipynb` for deployment.

Roadmap

Interlude: equality-constrained Newton's method

Before tackling the barrier subproblem, let's review the building block: **Newton's method for equality-constrained minimization**.

Problem.

$$\min_{\mathbf{x}\in\mathbb{R}^n} \; f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{h}(\mathbf{x}) = 0, \; \mathbf{h} : \mathbb{R}^n \to \mathbb{R}^p.$$

Lagrangian and KKT.

$$\mathcal{L}(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}).$$

First-order optimality (assuming LICQ) requires

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mathbf{y}) = \nabla f(\mathbf{x}) + J_h(\mathbf{x})^\top \mathbf{y} = \mathbf{0}, \qquad \mathbf{h}(\mathbf{x}) = \mathbf{0}.$$

That's $n + p$ nonlinear equations in $n + p$ unknowns $(\mathbf{x}, \mathbf{y})$.

Idea. Treat these $n + p$ equations as a root-finding problem and apply Newton's method to the *whole pair* $(\mathbf{x}, \mathbf{y})$ simultaneously — primal variables and multipliers.

This is not the only approach. Alternatives include: reduce to the tangent space of $\mathbf{h} = 0$ and run Newton on the reduced problem (null-space methods); penalty methods; augmented Lagrangian. Primal-dual Newton is the most direct and is what IPOPT and SQP solvers use.

Equality-constrained Newton: deriving the KKT system

Stack KKT into a residual vector $F(\mathbf{x}, \mathbf{y})$ in the unknown $\mathbf{z} = (\mathbf{x}, \mathbf{y})$:

$$F(\mathbf{z}) \;=\; \begin{pmatrix} \nabla f(\mathbf{x}) + J_h(\mathbf{x})^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \end{pmatrix} \;=\; \mathbf{0}.$$

Newton on this system. To solve $F(\mathbf{z}) = 0$, take Newton steps $\mathbf{z}^{k+1} = \mathbf{z}^k + \Delta\mathbf{z}$ where

$$\nabla F(\mathbf{z}^k)\, \Delta\mathbf{z} \;=\; -F(\mathbf{z}^k).$$

Compute the Jacobian ∇F. Each row of F contributes one row-block:

$$\partial_{\mathbf{x}}\left[\nabla f + J_h^\top \mathbf{y}\right] = \nabla^2 f(\mathbf{x}) + \sum_{j=1}^{p} y_j \nabla^2 h_j(\mathbf{x}) \;=:\; W, \qquad \partial_{\mathbf{y}}\left[\nabla f + J_h^\top \mathbf{y}\right] = J_h^\top.$$

$$\partial_{\mathbf{x}}[\mathbf{h}] = J_h, \qquad \partial_{\mathbf{y}}[\mathbf{h}] = 0.$$

So:

$$\nabla F = \begin{pmatrix} W & J_h^\top \\ J_h & 0 \end{pmatrix}.$$

The Newton KKT system.

$$\boxed{\begin{pmatrix} W(\mathbf{x}^k, \mathbf{y}^k) & J_h(\mathbf{x}^k)^\top \\ J_h(\mathbf{x}^k) & 0 \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \end{pmatrix} = -\begin{pmatrix} \nabla f(\mathbf{x}^k) + J_h(\mathbf{x}^k)^\top \mathbf{y}^k \\ \mathbf{h}(\mathbf{x}^k) \end{pmatrix}.}$$

Equivalent "full-step" form. Let $\mathbf{y}^+ := \mathbf{y}^k + \Delta\mathbf{y}$. Then (expand, cancel the $J_h^\top \mathbf{y}^k$ term on both sides):

$$\begin{pmatrix} W & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \mathbf{y}^+ \end{pmatrix} = \begin{pmatrix} -\nabla f(\mathbf{x}^k) \\ -\mathbf{h}(\mathbf{x}^k) \end{pmatrix}.$$

Both give the same $\Delta\mathbf{x}$; the second returns the *new* multiplier directly. (Nocedal–Wright uses this form.)

The saddle-point matrix and what it means

The matrix

$$K = \begin{pmatrix} W & J_h^\top \\ J_h & 0 \end{pmatrix} \in \mathbb{R}^{(n+p)\times(n+p)}$$

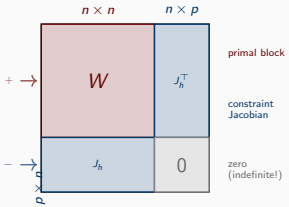
is called the **KKT matrix** or **saddle-point matrix**.

Key properties.

- **Symmetric** (since W is symmetric).
- **Indefinite**: both positive and negative eigenvalues (because of the 0 block). *Not* positive definite; Cholesky doesn't apply.
- **Inertia at solution**: under 2nd-order sufficient conditions + LICQ, K has exactly n_+, p_-, 0_0 eigenvalues: $\mathrm{inertia}(K) = (n, p, 0)$.
- **Nonsingular** iff J_h full row rank *and* W pos. def. on $\ker J_h$.

How it's solved in practice.

- Sparse symmetric indefinite $LDL^\top$ (Bunch–Kaufman pivoting).
- Industrial: **MA27**, **MA57** (HSL), **MUMPS**, **Pardiso**.
- One factorization + one triangular solve per Newton iteration.



2×2 **block form.** The 0 block kills positive-definiteness.

Preview. For (P_μ), plug the *barrier Hessian and gradient* into this same structure. IPOPT extends to a 3×3 block by treating **s** as separate variables (primal-dual upgrade, next).

Primal barrier method: the naïve algorithm

Path-following template:

1. Pick $\mu_0 > 0$ and a strictly interior $\mathbf{x}^{(0)}$.
2. For $k = 0, 1, 2, \ldots$:
   (a) Solve (approximately) (P_{μ_k}) by Newton's method, starting from $\mathbf{x}^{(k)}$.
   (b) Let $\mathbf{x}^{(k+1)}$ be the result.
   (c) Decrease μ: $\mu_{k+1} = \sigma\mu_k$ for some $\sigma \in (0, 1)$ (e.g. $\sigma = 0.1$).
3. Stop when $\mu_k < \epsilon$.

What Newton does on (P_μ): it's ordinary equality-constrained Newton (which you know) with the Lagrangian

$$\mathcal{L}_\mu(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) - \mu \sum_i \log(-g_i(\mathbf{x})) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}),$$

and KKT matrix built from $\nabla^2_x \mathcal{L}_\mu$ and $\nabla \mathbf{h}$.

Problem: the Hessian $\nabla^2 \Phi$ has entries scaling like $1/g_i^2$. Near the boundary these blow up $\Rightarrow$ ill-conditioning. Newton works but needs careful step-control.

Primal-dual interior point (what IPOPT does)

Instead of solving the barrier problem to convergence at each μ_k, **linearize the entire perturbed KKT system** and take *one* Newton step per outer iteration, then decrease μ.

Write the perturbed KKT system (equality constraints only, for clarity):

$$F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}) \;=\; \begin{pmatrix} \nabla f(\mathbf{x}) + J_g(\mathbf{x})^\top \mathbf{s} + J_h(\mathbf{x})^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \\ S\,\Sigma(\mathbf{x})\mathbf{1} - \mu\mathbf{1} \end{pmatrix} \;=\; 0,$$

where $S = \operatorname{diag}(s_1, \ldots, s_m)$ and $\Sigma(\mathbf{x}) = \operatorname{diag}(-g_1(\mathbf{x}), \ldots, -g_m(\mathbf{x}))$. The third block is the vector $\big(s_i \cdot (-g_i) - \mu\big)_i$ (perturbed complementarity).

Newton step: solve

$$\nabla F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}) \begin{pmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \\ \Delta\mathbf{s} \end{pmatrix} = -F_\mu(\mathbf{x}, \mathbf{y}, \mathbf{s}).$$

Key payoff. $(\mathbf{x}, \mathbf{s})$ and $(\mathbf{y}, \mathbf{s})$ are updated *simultaneously*. The complementarity equation is linearized like any other — no active-set decision needed.

What $\mu \to 0$ actually buys us: the duality gap

The quantity $S\Sigma\mathbf{1}$ — the vector with entries $s_i \cdot (-g_i(\mathbf{x}))$ — isn't just a residual. It *is* the duality gap.

LP in standard form. $\min \mathbf{c}^\top \mathbf{x}$ s.t. $A\mathbf{x} = \mathbf{b},\ \mathbf{x} \geq 0$. Dual: $\max \mathbf{b}^\top \mathbf{y}$ s.t. $A^\top \mathbf{y} + \mathbf{s} = \mathbf{c},\ \mathbf{s} \geq 0$. For any feasible $(\mathbf{x}, \mathbf{y}, \mathbf{s})$:

$$\mathbf{c}^\top \mathbf{x} - \mathbf{b}^\top \mathbf{y} \;=\; (A^\top \mathbf{y} + \mathbf{s})^\top \mathbf{x} - \mathbf{b}^\top \mathbf{y} \;=\; \mathbf{y}^\top A\mathbf{x} + \mathbf{s}^\top \mathbf{x} - \mathbf{b}^\top \mathbf{y} \;=\; \mathbf{s}^\top \mathbf{x}.$$

So the **duality gap equals** $\mathbf{s}^\top \mathbf{x} = \sum_i x_i s_i = \operatorname{tr}(XS) = \mathbf{1}^\top XS\mathbf{1}$.

On the central path. The perturbed complementarity says $x_i s_i = \mu$ for every i, so

$$\boxed{\operatorname{gap}(\mathbf{x}^\star(\mu), \mathbf{s}^\star(\mu)) \;=\; n\mu.}$$

Picture. As $\mu \downarrow 0$:

- Each complementarity product $x_i s_i$ is driven from $\mu \to 0$.
- Sum drops from $n\mu \to 0$: the primal and dual objectives squeeze together.
- Once gap $\leq \varepsilon$, we've solved to accuracy ε.

For general NLP ($g_i(\mathbf{x}) \leq 0$): the same story with slack $-g_i(\mathbf{x})$ replacing x_i. Gap at central-path point is $m\mu$ where m is the number of inequality constraints. Driving $\mu \to 0 =$ driving the duality gap to zero, one complementarity pair at a time.

Stopping criterion. IPOPT terminates when (scaled) $\|S\Sigma\mathbf{1}\|_\infty \leq \varepsilon_{\text{comp}}$ — the ∞-norm of complementarity products falls below a threshold. That's the same thing as the per-coordinate duality gap falling below tolerance.

The primal-dual Newton system

Writing the block structure of ∇F_μ:

$$\begin{pmatrix} W(\mathbf{x},\mathbf{y},\mathbf{s}) & J_h^\top & J_g^\top \\ J_h & 0 & 0 \\ -S\,J_g & 0 & \Sigma \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \\ \Delta\mathbf{s} \end{pmatrix} = - \begin{pmatrix} \nabla_{\mathbf{x}}\mathcal{L} \\ \mathbf{h} \\ S\,\Sigma\mathbf{1} - \mu\mathbf{1} \end{pmatrix}$$

where $W = \nabla^2 f + \sum s_i \nabla^2 g_i + \sum y_j \nabla^2 h_j$ is the Hessian of the (true, not barrier) Lagrangian, and $\Sigma = \operatorname{diag}(-g_i(\mathbf{x}))$.

Standard reduction: eliminate $\Delta\mathbf{s}$ via the third row, get a symmetric indefinite system in $(\Delta\mathbf{x}, \Delta\mathbf{y})$:

$$\begin{pmatrix} W + J_g^\top D J_g & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \end{pmatrix} = \text{rhs}, \qquad D = \Sigma^{-1} S = \operatorname{diag}\big(s_i/(-g_i)\big).$$

Factorize once per iteration. IPOPT uses MA27/MA57/MUMPS (sparse symmetric-indefinite solvers) for this step. The cost per iteration is dominated by this factorization.

From Lagrangian to Newton system: the logical chain

Step 1 — Lagrangian. Attach multipliers $\mathbf{y}$ (equalities) and $\mathbf{s} \geq 0$ (inequalities):

$$\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{s}) = f(\mathbf{x}) + \mathbf{y}^\top \mathbf{h}(\mathbf{x}) + \mathbf{s}^\top \mathbf{g}(\mathbf{x}).$$

Step 2 — KKT conditions from stationarity of $\mathcal{L}$ plus feasibility and complementarity:

$$\underbrace{\nabla_{\mathbf{x}}\mathcal{L} = \nabla f + J_g^\top \mathbf{s} + J_h^\top \mathbf{y} = 0}_{\text{stationarity}}, \quad \underbrace{\mathbf{h}(\mathbf{x}) = 0}_{\text{eq. feas.}}, \quad \underbrace{s_i(-g_i(\mathbf{x})) = 0}_{\text{complementarity}}, \quad \mathbf{s} \geq 0, \ \mathbf{g}(\mathbf{x}) \leq 0.$$

Step 3 — Perturb the hard equation. Replace $s_i(-g_i) = 0$ with $s_i(-g_i) = \mu$. Pack the three *equality-type* conditions into one nonlinear system in the unknown vector $\mathbf{z} = (\mathbf{x}, \mathbf{y}, \mathbf{s})$:

$$F_\mu(\mathbf{z}) \;=\; \begin{pmatrix} \nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{s}) \\ \mathbf{h}(\mathbf{x}) \\ s_i(-g_i(\mathbf{x})) - \mu \end{pmatrix} \;=\; \mathbf{0}.$$

(Inequalities $\mathbf{s} > 0$, $\mathbf{g}(\mathbf{x}) < 0$ are enforced *separately* by the fraction-to-boundary rule.)

Step 4 — Newton's method for a nonlinear system. To solve $F_\mu(\mathbf{z}) = 0$, iterate $\mathbf{z}^{k+1} = \mathbf{z}^k + \Delta\mathbf{z}$ where

$$\boxed{\nabla F_\mu(\mathbf{z}^k)\, \Delta\mathbf{z} \;=\; -\, F_\mu(\mathbf{z}^k).}$$

This is *the* matrix system. ∇F_μ is the **Jacobian** of the residual vector — a matrix whose (i, j) entry is $\partial F_i / \partial z_j$.

Step 5 — Why "block" structure? F_μ is naturally partitioned into three row-blocks (stationarity, equality, complementarity) and $\mathbf{z}$ into three column-blocks $(\mathbf{x}, \mathbf{y}, \mathbf{s})$. So ∇F_μ is a 3×3 grid of **blocks**:

$$\nabla F_\mu = \begin{pmatrix} \partial(\nabla_{\mathbf{x}}\mathcal{L})/\partial\mathbf{x} & \partial(\nabla_{\mathbf{x}}\mathcal{L})/\partial\mathbf{y} & \partial(\nabla_{\mathbf{x}}\mathcal{L})/\partial\mathbf{s} \\ \partial\mathbf{h}/\partial\mathbf{x} & \partial\mathbf{h}/\partial\mathbf{y} & \partial\mathbf{h}/\partial\mathbf{s} \\ \partial[S(-\mathbf{g}) - \mu]/\partial\mathbf{x} & \partial[\cdot]/\partial\mathbf{y} & \partial[\cdot]/\partial\mathbf{s} \end{pmatrix}.$$

The next slide fills in these nine entries. Most are zero or immediate; only the Hessian-of-Lagrangian block W requires work.

Deriving the Newton step block by block

Let J_g (rows $\nabla g_i^\top$), J_h be Jacobians; $S = \operatorname{diag}(s_i)$, $\Sigma(\mathbf{x}) = \operatorname{diag}(-g_i(\mathbf{x}))$ ($\succ 0$ interior). Perturbed-KKT residuals:

$$F_\mu = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \end{pmatrix} = \begin{pmatrix} \nabla f + J_g^\top \mathbf{s} + J_h^\top \mathbf{y} \\ \mathbf{h}(\mathbf{x}) \\ S\Sigma\mathbf{1} - \mu\mathbf{1} \end{pmatrix}, \quad [S\Sigma\mathbf{1}]_i = s_i(-g_i).$$

Differentiate each block. $\partial_{\mathbf{x}} r_1 = \nabla^2 f + \sum s_i \nabla^2 g_i + \sum y_j \nabla^2 h_j =: W$; $\partial_{\mathbf{y}} r_1 = J_h^\top$, $\partial_{\mathbf{s}} r_1 = J_g^\top$. $\partial_{\mathbf{x}} r_2 = J_h$. $\partial_{\mathbf{x}} r_3 = -SJ_g$, $\partial_{\mathbf{s}} r_3 = \Sigma$.

$$\nabla F_\mu = \begin{pmatrix} W & J_h^\top & J_g^\top \\ J_h & 0 & 0 \\ -SJ_g & 0 & \Sigma \end{pmatrix}.$$

Reduce to symmetric indefinite. From row 3 with $\Sigma \succ 0$: $\Delta\mathbf{s} = \underbrace{\Sigma^{-1}S}_{=:D} J_g \Delta\mathbf{x} - \Sigma^{-1} r_3$. Substitute:

$$\boxed{\begin{pmatrix} W + J_g^\top D J_g & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} \Delta\mathbf{x} \\ \Delta\mathbf{y} \end{pmatrix} = -\begin{pmatrix} r_1 - J_g^\top \Sigma^{-1} r_3 \\ r_2 \end{pmatrix}.}$$

IPOPT factorizes this with MA27/MA57/MUMPS. Recover $\Delta\mathbf{s}$ from the reduction.

$D = \operatorname{diag}(s_i/(-g_i))$ is the *barrier-weighted* active-constraint penalty: as $-g_i \to 0$, $D_{ii} \to \infty$, enforcing the constraint.

Worked example: one primal-dual Newton iteration

Problem. $\min x^2$ s.t. $x \geq 1$. Rewrite $g(x) = 1 - x \leq 0$. Optimum $(x^\star, s^\star) = (1, 2)$.

Central path. Barrier stationarity $2x - \mu/(x-1) = 0$ gives $x(\mu) = \frac{1+\sqrt{1+2\mu}}{2}$, $s(\mu) = 2x(\mu)$. As $\mu \downarrow 0$: $(x, s) \to (1, 2)$.✓

Ingredients. $\nabla f = 2x$, $J_g = -1$, $W = 2$, $\Sigma = x - 1$. Residuals: $r_1 = 2x - s$, $r_3 = s(x-1) - \mu$. Jacobian:

$$\nabla F_\mu = \begin{pmatrix} W & J_g^\top \\ -SJ_g & \Sigma \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ s & x-1 \end{pmatrix}.$$

Iterate 0. $(x^0, s^0) = (2, 2)$, $\mu_0 = 1$, $r_1 = 2$, $r_3 = 1$. Solve $\left(\begin{smallmatrix} 2 & -1 \\ 2 & 1 \end{smallmatrix}\right)\left(\begin{smallmatrix} \Delta x \\ \Delta s \end{smallmatrix}\right) = -\left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \Rightarrow \Delta x = -0.75$, $\Delta s = 0.5$.

Fraction-to-boundary ($\tau = 0.995$): $1 - 0.75\alpha \geq 0.005 \Rightarrow \alpha \leq 1.327$; slack always OK. Full step $\alpha = 1 \Rightarrow \boxed{(x^1, s^1) = (1.25, 2.5)}$.

Progress. $r_1 = 0$ exact; $s^1(x^1 - 1) = 0.625$ (was 1). Decrease $\mu_1 = 0.25$; repeat.

Iterate 1 at $(1.25, 2.5)$, $\mu = 0.25$: $r_3 = 0.375$. Solve $\Rightarrow \Delta x = -0.125$, $\Delta s = -0.25$. $(x^2, s^2) = (1.125, 2.25)$.

Central-path at $\mu = 0.25$: $(1.112, 2.225)$. Within 0.025 — converging to $(1, 2)$. ✓

Roadmap

2D worked example: J_g, W, and the 3×3 Jacobian by hand

Problem. $\min f(\mathbf{x}) = x_1^2 + x_2^2$ s.t. $g_1(\mathbf{x}) = 1 - x_1 \le 0$, $g_2(\mathbf{x}) = 1 - x_2 \le 0$. Optimum $\mathbf{x}^\star = (1,1)$, $\mathbf{s}^\star = (2,2)$.

Ingredients (dimensions: $n = 2$, $m = 2$, $p = 0$).

$$\nabla f(\mathbf{x}) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}, \quad J_g(\mathbf{x}) = \begin{pmatrix} \nabla g_1^\top \\ \nabla g_2^\top \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}, \quad W = \nabla^2 f + \sum s_i \nabla^2 g_i = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix},$$

$$\Sigma(\mathbf{x}) = \operatorname{diag}(-g_i) = \operatorname{diag}(x_1 - 1,\ x_2 - 1), \qquad S = \operatorname{diag}(s_1, s_2).$$

Residuals at current $(\mathbf{x}, \mathbf{s})$:

$$r_1 = \nabla f + J_g^\top \mathbf{s} = \begin{pmatrix} 2x_1 - s_1 \\ 2x_2 - s_2 \end{pmatrix}, \qquad r_3 = S\Sigma\mathbf{1} - \mu\mathbf{1} = \begin{pmatrix} s_1(x_1 - 1) - \mu \\ s_2(x_2 - 1) - \mu \end{pmatrix}.$$

4×4 **Jacobian** ($n + m = 4$; $p = 0$, no equalities):

$$\nabla F_\mu = \begin{pmatrix} W & J_g^\top \\ -SJ_g & \Sigma \end{pmatrix} = \begin{pmatrix} 2 & 0 & -1 & 0 \\ 0 & 2 & 0 & -1 \\ s_1 & 0 & x_1 - 1 & 0 \\ 0 & s_2 & 0 & x_2 - 1 \end{pmatrix}.$$

Iterate at $(\mathbf{x}, \mathbf{s}) = (2,2,2,2)$, $\mu = 1$. $r_1 = (2,2)$, $r_3 = (2 \cdot 1 - 1,\ 2 \cdot 1 - 1) = (1,1)$. The 4×4 Jacobian is block-diagonal into two identical 2×2 systems (problem separates!):

$$\begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_i \\ \Delta s_i \end{pmatrix} = -\begin{pmatrix} 2 \\ 1 \end{pmatrix} \ \forall i \ \Rightarrow \ \Delta x_i = -0.75,\ \Delta s_i = 0.5.$$

Full step $\Rightarrow \mathbf{x}^1 = (1.25, 1.25)$, $\mathbf{s}^1 = (2.5, 2.5)$. Same arithmetic as the 1D example, *twice* — the separability makes it transparent.

Step acceptance: fraction-to-the-boundary rule

After computing $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{s})$, we need to step without leaving the strict interior.

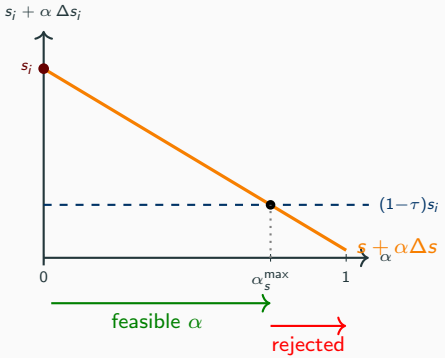
Fraction-to-the-boundary with $\tau_k \in (0,1)$ (e.g. $\tau_k = 0.995$):

$$\alpha_s^{\max} = \max\{\alpha \in (0,1] \,:\, s_i + \alpha \Delta s_i \geq (1-\tau_k) s_i \;\forall i\}.$$

(Max over α, but inequality must hold for *all* i; equivalent to the per-component $\min_i$.) Similar bound for $\mathbf{x}$ against $g_i(\mathbf{x}) \leq 0$.

Then line-search on a merit function / filter (we'll get to the filter in Act III) starting from $\alpha^{\max}$ downward.

- Keeps $\mathbf{x}$ in strict interior and $\mathbf{s} > 0$ throughout.
- Stays on or near the central path.
- $\tau_k \to 1$ as $\mu_k \to 0$ — allow longer steps near optimality.



The longest α we allow before the slack would drop below $(1-\tau)s_i$.

Fraction-to-boundary by hand: numerical example

Setup. Current dual multipliers $\mathbf{s} = (1.0, 0.5, 2.0)$ (each must stay > 0). Newton step $\Delta\mathbf{s} = (-0.2,\ -0.6,\ +0.4)$. Use $\tau = 0.995$, so floor $(1-\tau)\mathbf{s} = (0.005,\ 0.0025,\ 0.01)$.

Per-component bound. Solve $s_i + \alpha \Delta s_i \geq (1-\tau) s_i$ for each i:

$$\alpha_i = \begin{cases} \dfrac{-s_i(1-(1-\tau))}{\Delta s_i} = \dfrac{-\tau s_i}{\Delta s_i} & \text{if } \Delta s_i < 0, \\ +\infty & \text{if } \Delta s_i \geq 0 \end{cases}$$

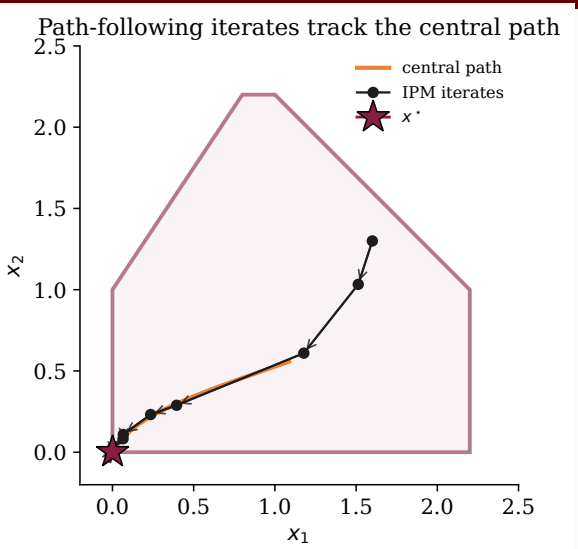
i	s_i	Δs_i	$(1-\tau)s_i$	sign	α_i
1	1.0	-0.2	0.005	< 0	$-0.995(1.0)/(-0.2) = \mathbf{4.975}$
2	0.5	-0.6	0.0025	< 0	$-0.995(0.5)/(-0.6) = \mathbf{0.829}$
3	2.0	$+0.4$	0.01	≥ 0	$+\infty$

Combine. $\alpha_s^{\max} = \min(1,\ \min_i \alpha_i) = \min(1,\ 0.829) = \mathbf{0.829}$. Component i=2 is *binding*: s_2 would hit its floor at $\alpha = 0.829$. Take the step, and verify:

$$\mathbf{s} + 0.829\,\Delta\mathbf{s} \ = \ (0.834,\ 0.003,\ 2.332), \qquad s_2 = 0.003 \geq 0.0025 \ \checkmark$$

The $\min_i$ is essential: one tight component throttles the whole step. A good starting point has *balanced* slacks so no component dominates $\alpha^{\max}$.

Picture: iterates tracking the central path



Primal-dual Newton + fraction-to-boundary, started from an off-path interior point.

- Early iterates *swing* toward the central path (centering).
- Once on the path, each step roughly halves μ and slides along the curve.
- Short-step analysis: polynomial complexity $O(\sqrt{n}\log(1/\varepsilon))$ Newton iterations (n = number of complementarity pairs).

Why Newton? Local quadratic convergence

The inner solver at each μ is Newton's method on the perturbed KKT system $F_\mu = 0$. Why is that the right choice?

Theorem [Nocedal–Wright Thm. 11.2, informally]. If ∇F_μ is Lipschitz near a solution $\mathbf{z}^\star(\mu)$ and is nonsingular there, then Newton's method started close enough to $\mathbf{z}^\star(\mu)$ satisfies

$$\|\mathbf{z}^{k+1} - \mathbf{z}^\star\| \;\le\; C\,\|\mathbf{z}^k - \mathbf{z}^\star\|^2.$$

That is, the *number of correct digits doubles per iteration* once you are close.

Why this matters for IPM:

- At each μ, we don't need to solve exactly — a few Newton steps drops the residual to $O(\mu)$.
- Decreasing μ by a constant factor and warm-starting from the previous $\mathbf{z}^\star(\mu)$ keeps us inside the quadratic-convergence region.
- Short-step analysis (next slide) turns this into a provable polynomial bound.

The alternative. First-order methods on the barrier problem converge linearly — you'd need $\log(1/\varepsilon)$ *per* μ, times $\log(1/\varepsilon)$ outer updates. Newton wins because inner cost is constant.

Short-step IPM: polynomial-time complexity

Setting. LP-like case: f linear, only bound constraints $x \geq 0$. Let n = number of such nonneg constraints (= number of complementarity pairs). Here $X = \operatorname{diag}(x_i)$, $S = \operatorname{diag}(s_i)$.

Proximity measure.

$$\delta(\mathbf{x}, \mathbf{s}; \mu) \;=\; \left\| \frac{XSe}{\mu} - e \right\|_2, \qquad X = \operatorname{diag}(x_i),\; S = \operatorname{diag}(s_i).$$

$\delta = 0 \Leftrightarrow$ on the central path. A **neighborhood** $\mathcal{N}_2(\theta) = \{\delta \leq \theta\}$ is a tube around the path.

Short-step path-following algorithm.

1. Start with $(\mathbf{x}^0, \mathbf{s}^0) \in \mathcal{N}_2(\theta)$, $\theta \in (0, 1)$.
2. At iteration k: set $\mu_{k+1} = (1 - \sigma/\sqrt{n})\mu_k$ for a small $\sigma > 0$.
3. Take one full Newton step on the perturbed KKT at μ_{k+1}.
4. Analysis shows the new iterate remains in $\mathcal{N}_2(\theta)$.

Theorem [Nocedal–Wright Thm. 14.4; Renegar 1988]. After

$$K \;=\; O\Big(\sqrt{n}\,\log\frac{n\mu_0}{\varepsilon}\Big)$$

iterations, the duality gap $n\mu_K \leq \varepsilon$.

Intuition. Each iteration:

- Shrinks μ by a factor $1 - \sigma/\sqrt{n}$ (slow, because $\sqrt{n}$ in denominator).
- Needs only *one* Newton step (because of quadratic convergence within the neighborhood).

Product: total Newton steps scale like $\sqrt{n}\log(1/\varepsilon)$ — **polynomial in** n **and** $\log(1/\varepsilon)$.

Historical context. This is the complexity result that broke the simplex-is-exponential-worst-case barrier for LP. Karmarkar (1984) — projective-scaling IPM, $O(n^{3.5}L)$. Renegar (1988) — path-following IPM, $O(\sqrt{n}L)$. Nesterov–Nemirovski (1994) — self-concordant barriers, general convex cones.

How the polynomial complexity bound is proved (Wright)

Tying into what we've seen. All three μ-updates we've written down have the form

$$\mu_{k+1} \;\le\; \left(1 - C n^{-\rho}\right) \mu_k$$

for some rate exponent $\rho \ge \frac{1}{2}$:

Method	Rate	ρ
Short-step (previous slide)	$1 - \sigma/\sqrt{n}$	$\frac{1}{2}$
Long-step / predictor–corrector	$1 - C/n$	1
Mehrotra (worst case)	$1 - C/n$	1

Telescoping argument. To drive $\mu_K \le \varepsilon \mu_0$:

$$\begin{aligned} \mu_K \le \varepsilon \mu_0 &\;\Leftarrow\; \left(1 - C n^{-\rho}\right)^K \le \varepsilon \\ &\;\Leftarrow\; K\left(-C n^{-\rho}\right) \le \log \varepsilon \qquad \text{by } \log(1+t) \le t \\ &\;\Leftrightarrow\; K \ge n^{\rho} \left|\log \varepsilon\right| / C. \end{aligned}$$

Conclusion.

$$\boxed{K \;=\; O\!\left(n^{\rho} \left|\log \varepsilon\right|\right)} \text{ iterations.}$$

Short-step ($\rho = \frac{1}{2}$) recovers the celebrated $O(\sqrt{n}\log(1/\varepsilon))$ bound — the same shape as the Nesterov–Nemirovski master theorem $O(\sqrt{\nu}\log(1/\varepsilon))$ with n in the role of the barrier parameter for the LP orthant.

ρ here is the *rate exponent*, not the self-concordance parameter ν. Despite the name collision in the literature, they play the same role: they both control how many outer iterations you need.

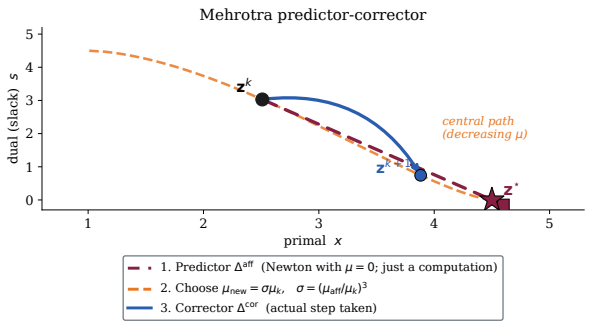
Mehrotra's predictor-corrector — the big idea

Why bother? Short-step shrinks μ by only $1 - \sigma/\sqrt{n}$ per iteration — slow. Industrial solvers take **long steps** with *adaptive* μ: let the *local geometry* tell us how aggressive to be.

Mehrotra (1992): three steps per iteration from $\mathbf{z}^k = (\mathbf{x}^k, \mathbf{y}^k, \mathbf{s}^k)$.

1. **Predictor** — a *trial computation*. Solve the Newton system with $\mu = 0$ on the RHS; this is the step that aims straight at the true KKT point. Look at where it would land, but *don't actually move there*.

2. **Pick new μ.** Based on how much progress the predictor *would* have



Key takeaway. The predictor is a *free measurement* of how good the local geometry is; the corrector is where the algorithm actually spends a step. Two Newton solves per outer iteration, one target.

What is μ_{aff}? The math behind the σ-rule

Definition. Let $\Delta \mathbf{z}^{\text{aff}}$ be the **predictor direction** — the Newton step with $\mu = 0$ on the complementarity row. Let $\alpha^{\text{aff}} \in (0, 1]$ be the longest step that keeps $(\mathbf{x}, \mathbf{s})$ strictly interior (fraction-to-boundary). Then:

$$\mu_{\text{aff}} \;:=\; \frac{(\mathbf{s}^k + \alpha^{\text{aff}} \Delta \mathbf{s}^{\text{aff}})^\top \big(-\mathbf{g}(\mathbf{x}^k + \alpha^{\text{aff}} \Delta \mathbf{x}^{\text{aff}})\big)}{m}.$$

In words: μ_{aff} is the *average complementarity* at the hypothetical point you'd reach if you took the predictor step. "The μ we would have achieved if we'd taken the predictor."

Compare with the current μ_k. At the current iterate, $\mu_k = \mathbf{s}^{k\top}(-\mathbf{g}(\mathbf{x}^k))/m$. The **centrality ratio** is

$$\sigma \;:=\; \left(\frac{\mu_{\text{aff}}}{\mu_k}\right)^3 \in [0, 1].$$

Reading the heuristic.

- Predictor *aggressive* — gets near the optimum, $\mu_{\text{aff}} \ll \mu_k$, so $\sigma \approx 0$. Target $\mu_{\text{new}} = \sigma \mu_k \approx 0$: set an ambitious target.
- Predictor *feeble* — barely moves, $\mu_{\text{aff}} \approx \mu_k$, so $\sigma \approx 1$. Target $\mu_{\text{new}} \approx \mu_k$: don't overreach.

Why the cube? Empirical tuning by Mehrotra. A cube is more aggressive than a linear interpolation (so the solver actually takes advantage of a good predictor) but still polynomial, so small μ_{aff} doesn't run away.

Practical behavior. On well-behaved NLPs this gets IPOPT to convergence in 20–100 Newton iterations regardless of problem size — HS071 takes only 6. Two linear solves per iteration (predictor + corrector), both with the *same* factored matrix, so the second one is nearly free.

Roadmap

Coding Break #3 — from CVXPY to IPOPT

CVXPY handed our 2D LP to a conic IPM. Now let's write the *same* problem in Pyomo and solve it with **IPOPT**.

Why do this twice?

- CVXPY is a modeling language for *convex* problems. It proves convexity (DCP) and reformulates to a cone.
- Pyomo is a modeling language for *general* NLP. It doesn't check convexity; IPOPT doesn't care either.
- Both land at $(1, 0)$ for our LP. But IPOPT gives raw access to its log and final diagnostics — the window into the primal-dual Newton machinery we just derived in Act II.

Notebook: `part1_solvers/04_ipopt_pyomo_cyipopt.ipynb`

What we'll watch:

1. The iteration log, column by column (`iter`, `inf_pr`, `lg(mu)`, ...).

Pyomo + IPOPT: the 2D LP, by the book

```
import pyomo.environ as pyo

m = pyo.ConcreteModel()
m.x1 = pyo.Var(domain=pyo.NonNegativeReals)
m.x2 = pyo.Var(domain=pyo.NonNegativeReals)
m.obj = pyo.Objective(expr = -m.x1, sense=pyo.minimize)
m.c1  = pyo.Constraint(expr = m.x1 + m.x2 <= 1)

m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)

solver = pyo.SolverFactory("ipopt")
results = solver.solve(m, tee=True)       # tee=True prints the log

print("x1 =", pyo.value(m.x1))
print("x2 =", pyo.value(m.x2))
print("dual of c1 =", m.dual[m.c1])
print("termination:", results.solver.termination_condition)
```


Expect convergence in 5–10 iterations with `EXIT: Optimal Solution Found.` at

Roadmap

Do we need a Phase I? Primal-dual initialization

Recall simplex. Classical simplex requires a *basic feasible solution* to start. If you don't have one, you run **Phase I**: a separate LP that constructs a feasible vertex (e.g. Big-M or the two-phase method).

Modern primal-dual IPM — no Phase I needed. You only need

$$\mathbf{s}^0 > 0 \quad \text{and} \quad -g_i(\mathbf{x}^0) > 0 \ \forall i \qquad \text{(strict "interior" of bounds/slacks)}$$

The *equality* constraints $\mathbf{h}(\mathbf{x}) = 0$ need **not** hold initially — the Newton system drives all three residuals r_1, r_2, r_3 to zero *simultaneously*. This is the **infeasible-start primal-dual IPM** (Kojima–Megiddo–Mizuno 1993; Wright 1997, Ch. 6).

Three ways different solvers handle "Phase I":

- **Infeasible-start**: Start anywhere with $\mathbf{s}, -g > 0$. Let Newton handle it. *IPOPT, KNITRO, LOQO.*
- **Homogeneous self-dual (HSD) embedding**: Embed the LP/conic program in a bigger problem with a trivial strictly feasible starting point; the solution reveals optimal, primal-infeasible, or dual-infeasible. *MOSEK, ECOS, SeDuMi, SCS.*
- **Big-M / penalty**: Classical. Rarely used in modern IPM implementations.

Moral. For IPOPT, you don't construct a feasible point *up front* — you just need the *strict interior* of the bounds and inequality constraints. Equality feasibility is part of what Newton solves.

How IPOPT actually initializes the iterates

Starting from your $\mathbf{x}^0$ (and optionally $\mathbf{y}^0, \mathbf{s}^0$), IPOPT performs four steps:

1. Slack reformulation. Every inequality $g_i(\mathbf{x}) \le 0$ is internally rewritten as $g_i(\mathbf{x}) + s_i = 0,\ s_i \ge 0$. IPOPT's internal problem has only equalities + bound constraints.

2. Push-to-interior for bounds. Any x_i^0 at or outside $[x_L, x_U]$ is shifted strictly inside by a small fraction (options `bound_push`, `bound_frac`, defaults 10^{-2}). Slacks start at $s_i^0 = \max(1, |g_i(\mathbf{x}^0)|)$.

3. Least-squares multiplier estimate. With $\mathbf{x}^0, \mathbf{s}^0$ fixed, IPOPT solves for $\mathbf{y}^0$ to minimize the initial dual residual $\|\nabla f + J_g^\top \mathbf{s}^0 + J_h^\top \mathbf{y}\|_2$ (option `least_square_init_duals yes`). Falls back to $\mathbf{y}^0 = 0$ if ill-conditioned.

4. Restoration phase (on-demand Phase I). If the filter line search cannot accept any step, IPOPT temporarily minimizes a feasibility measure $\|\mathbf{h}(\mathbf{x})\|_1 + \|\mathbf{g}(\mathbf{x}) + \mathbf{s}^g\|_1 + \frac{\zeta}{2}\|D(\mathbf{x} - \mathbf{x}_R)\|_2^2$ (here $\mathbf{s}^g$ are the slack variables on inequalities — not the dual multipliers) to pull the iterate back toward feasibility, then resumes. *This* is IPOPT's Phase I — but it fires on demand, not at the start.

A good starting point saves iterations. In Pyomo: `pyo.Var(initialize=value)` or `m.x[i].set_value(v)`. Supply warm-start multipliers when re-solving perturbed problems (option `warm_start_init_point yes`).

Why a globalization step is needed

The Newton step $\Delta \mathbf{z}$ is a *local* object — it's the right answer in a neighborhood of the central-path point, but not necessarily far away.

Failure modes without globalization:

- Full Newton step increases the objective dramatically.
- Full Newton step leaves the strict interior (partially handled by fraction-to-boundary, but still possible for equality violation).
- Full Newton step on a nonconvex problem lands at a non-descent direction.
- Full step oscillates between points of equal "merit" (cycling).

Classical solution: a merit function.

$$\phi_\pi(\mathbf{x}) \;=\; f(\mathbf{x}) \;+\; \pi \,\|\mathbf{h}(\mathbf{x})\|_1, \qquad \pi > 0$$

and do backtracking line search on ϕ_π.

Problem: choosing π.

- π too small: solver tolerates infeasibility, drifts away from $\{h = 0\}$.
- π too large: rejects steps that temporarily worsen $\|h\|$, stalling progress.
- Good choice depends on current iterate *and* step — no universal rule.

The Maratos effect: when merit functions kill Newton

Maratos (1978). There exist equality-constrained problems where the *full* Newton step (with superlinear convergence) is rejected by every ℓ_1 merit function ϕ_π for every π.

Toy example. Minimize $f(x_1, x_2) = 2(x_1^2 + x_2^2 - 1) - x_1$ subject to $h(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0$ (the unit circle). Optimum at $(1, 0)$.

From a point on the circle near $(1, 0)$, the Newton step is tangential and overshoots the circle slightly. Then:

- $\|h\|$ *increases* (step leaves circle).
- f decreases only to second order.
- $\phi_\pi = f + \pi\|h\|$ *increases* for every $\pi > 0 \Rightarrow$ step rejected.

Cost of rejection. Line search shrinks α, loses quadratic convergence, solver slows to linear rate.

Classical fixes.

- Second-order correction (Fletcher): add a correction term after the Newton step.
- Watchdog technique: accept non-monotone iterates under a "relaxation" rule.
- **Filter methods** (Fletcher–Leyffer 2002): treat f and $\|h\|$ as separate objectives. No penalty parameter.

Wächter–Biegler filter: the big idea

Two objectives, not one.

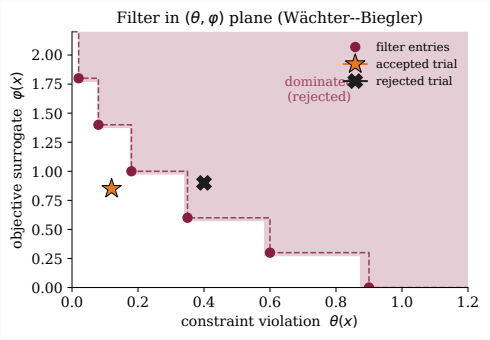
- Barrier objective $\varphi(\mathbf{x}) = f(\mathbf{x}) - \mu \sum \log(-g_i(\mathbf{x}))$
- Equality violation $\theta(\mathbf{x}) = \|\mathbf{h}(\mathbf{x})\|_1$

Filter $\mathcal{F}$. A set of pairs (θ', φ') from past iterates. A new point $\mathbf{x}^+$ is *acceptable to* $\mathcal{F}$ if, for every $(\theta', \varphi') \in \mathcal{F}$:

$$\theta(\mathbf{x}^+) \le (1 - \gamma_\theta)\theta' \quad \textbf{or} \quad \varphi(\mathbf{x}^+) \le \varphi' - \gamma_\varphi\, \theta(\mathbf{x}^+).$$

Reading. Accept if $\mathbf{x}^+$ is strictly better than *every* filter entry, in *at least one* of the two dimensions (with margin $\gamma_\theta, \gamma_\varphi \approx 10^{-5}$).

No penalty parameter anywhere. The filter is *the* acceptance criterion.



The shaded region is dominated — rejected. Everything else is acceptable.

Origin. Filter methods for NLP: Fletcher–Leyffer *Math. Prog.* 2002 (SQP setting). Wächter–Biegler *SIAM J. Optim.* 2005 adapted them to IPOPT with the barrier objective φ.

Filter mechanics: when to augment, f-type vs h-type

Step 1 — Compute trial point. $\mathbf{x}^+ = \mathbf{x}^k + \alpha \Delta \mathbf{x}$ starting from $\alpha = \alpha^{\max}$ (fraction-to-boundary).

Step 2 — Check acceptance to filter. If $(\theta(\mathbf{x}^+), \varphi(\mathbf{x}^+))$ is dominated, shrink $\alpha \to \alpha/2$ and retry.

Step 3 — If accepted, classify the iteration.

- **f-type** iteration: the Newton direction predicts sufficient decrease in φ (switching condition holds), *and* φ actually decreased. Treat as a "normal" optimization step — **do not augment** the filter.
- **h-type** iteration: otherwise. The step reduces θ but not φ. **Augment** $\mathcal{F}$ by adding $(\theta(\mathbf{x}^k), \varphi(\mathbf{x}^k))$.

Why this asymmetry? We want two forms of progress:

- On *good* (f-type) steps: don't clutter the filter — avoid locking ourselves out of this region later.
- On *repair* (h-type) steps: lock out the current (θ, φ) pair to guarantee we don't revisit.

Switching condition (schematic): f-type if the Armijo-type inequality $\varphi(\mathbf{x}^+) \leq \varphi(\mathbf{x}^k) - \eta\alpha \nabla\varphi^\top \Delta \mathbf{x}$ holds AND $\theta(\mathbf{x}^k)$ is small enough that we can trust $\nabla\varphi$ as a descent direction on the nonlinear objective.

Restoration phase and convergence guarantees

What if every $\alpha > 0$ along $\Delta\mathbf{x}$ is rejected by $\mathcal{F}$?

This can happen if the current direction is incompatible with the filter. Wächter–Biegler enter a *restoration phase*: temporarily abandon optimality and chase feasibility alone.

Restoration subproblem. Solve

$$\min_{\mathbf{x}} \ \|\mathbf{h}(\mathbf{x})\|_2^2 \ + \ (\text{regularization})$$

using the same interior-point machinery, until $\theta(\mathbf{x})$ is small enough that the trial point is again acceptable to the filter. Then return to the main IPOPT loop.

Theorem [Wächter–Biegler 2005]. Under standard regularity assumptions (LICQ at limit points, second-order sufficient conditions), the algorithm produces iterates such that *either*

- a subsequence $\{\mathbf{x}^k\}$ converges to a first-order KKT point of the original NLP, *or*
- the restoration phase detects local infeasibility (no KKT point exists locally).

Why no cycling? Each filter augmentation carves out a neighborhood of (θ', φ') permanently. In finitely many iterations either we accumulate finitely many augmentations (and then stay in an f-type region, where Newton + Armijo works) or we detect infeasibility.

In practice. Restoration fires rarely on well-posed NLPs. When it does, IPOPT reports it in the log (the "R" line prefix). If restoration cannot reduce θ, IPOPT terminates with "Converged to a point of local infeasibility".

Inertia correction: dealing with nonconvexity

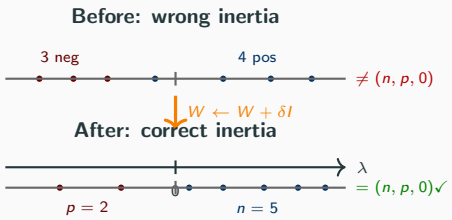
For convex problems, the KKT matrix is positive-definite on the null space of $J_h \Rightarrow$ Newton direction is descent. For **nonconvex** problems, W can have wrong-signed eigenvalues, and the Newton direction can fail to be a descent direction.

IPOPT's fix: inertia correction. After factorizing the KKT matrix, check that inertia is $(n_x, n_h, 0)$. If not, add a multiple of the identity to W:

$$W \leftarrow W + \delta I,$$

increasing δ until the inertia is correct. Search over $\delta \in \{10^{-8}, 10^{-4}, 10^{-2}, \dots\}$.

This is *regularization inside the solver*, not a user-facing knob. It's why IPOPT can handle nonconvex NLPs and return local KKT points — though of course not global ones.



Eigenvalues of K. Shift W until sign pattern matches target.

Convex? Nonconvex? IPOPT doesn't care — and that should worry you

IPOPT treats every NLP the same way. The algorithm never asks "is f convex?" or "is $\{g_i \le 0\}$ a convex set?". It just:

- Builds the perturbed KKT system.
- Factorizes $K = \begin{pmatrix} W + J_g^\top D J_g & J_h^\top \\ J_h & 0 \end{pmatrix}$ with inertia correction if needed.
- Takes a Newton step filtered by the filter line search.
- Decreases μ.

This is a *feature* — no solver-selection step, no convexity check — and a *trap*.

What IPOPT actually guarantees (Wächter–Biegler 2005). Under LICQ and second-order sufficient conditions at limit points, a subsequence of iterates converges to a point satisfying the **first-order KKT conditions** of the NLP *or* the algorithm detects local infeasibility.

What IPOPT does not guarantee:

- Global optimality (obviously, in nonconvex problems).
- That the returned point is a local *minimum* — it could be a saddle point or maximum that happens to satisfy first-order KKT.
- That multi-starts from different $\mathbf{x}^{(0)}$ will agree. On nonconvex problems **they usually don't**.
- That the "objective value" reported is comparable to another run with a different tolerance.

How IPOPT survives nonconvexity:

- **Inertia correction** (previous slide): force K to have correct inertia $(n_x, n_h, 0)$ — makes the direction a descent direction for the filter merit, even if W is not positive definite.
- **Filter line search**: accepts nonmonotone steps, avoids cycling in indefinite regions.
- **Restoration phase**: when no descent direction on the barrier objective exists, chase feasibility alone.
- **Bound relaxation**: $\mathbf{x}_L - \epsilon \le \mathbf{x} \le \mathbf{x}_U + \epsilon$ internally, to avoid artificial ill-conditioning.

Numerical accuracy: when IPOPT "cheats"

IPOPT doesn't return exact solutions. It returns iterates where a *scaled* KKT residual is below a tolerance:

$$\max\left(\frac{\|\nabla_{\mathbf{x}}\mathcal{L}\|_\infty}{s_d},\ \|\mathbf{h}\|_\infty,\ \frac{\|S\Sigma\mathbf{1}\|_\infty}{s_c}\right) \le \texttt{tol} \quad (\text{default } 10^{-8}).$$

with auxiliary tolerances `constr_viol_tol`, `dual_inf_tol`, `compl_inf_tol`. The scales s_d, s_c can *inflate* the tolerated residual when multipliers are large.

Daniel Bienstock's favorite example. On certain nonconvex polynomial NLPs, IPOPT reports "Solve_Succeeded" at a point where the *raw* (unscaled) constraint residual $\|\mathbf{h}(\mathbf{x})\|$ is much larger than the user expects — sometimes orders of magnitude larger than `tol`. The solver hasn't lied: by its own scaled metric, it converged. But the returned $\mathbf{x}$ isn't actually a feasible point of the stated problem. If you're using IPOPT to get a *bound* for a MINLP or a polynomial optimization problem, this "solution" can produce a bogus objective value that beats the true global optimum — IPOPT "cheats" by being slightly infeasible.

Trust but verify. After every IPOPT solve, **independently check**:

- $\|\mathbf{h}(\mathbf{x}^\star)\|_\infty$ and $\max_i g_i(\mathbf{x}^\star)$ computed at full precision — not what the solver reports.
- Do they meet *your* feasibility requirement (not IPOPT's scaled internal one)?
- Is the objective consistent with a neighboring feasible point?
- If multi-starting, how much do solutions disagree?

Mitigations in practice.

- Tighten: `tol=1e-12`, `constr_viol_tol=1e-10`, `bound_relax_factor=0`.
- Scale your problem yourself; don't rely on IPOPT's autoscaling for ill-conditioned constraints.
- For global bounds: use a global solver (BARON, SCIP, Gurobi's nonconvex QP/MIQCP mode). Never quote IPOPT output as a global bound.
- Post-solve projection onto the constraint set if you need strictly feasible iterates.

The IPOPT algorithm, assembled

High-level IPOPT iteration

1. **Initialize** $(\mathbf{x}^0, \mathbf{y}^0, \mathbf{s}^0)$ in the strict interior; μ_0; filter $\mathcal{F} = \emptyset$.
2. **Outer loop on μ:** While $\mu > \epsilon$:
   - 2.1 **Inner loop:** while KKT residual at current μ is not small:
     - 2.1.1 Factor the KKT matrix; *inertia correction* if needed.
     - 2.1.2 Solve for $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{s})$.
     - 2.1.3 Compute $\alpha^{\max}$ via fraction-to-the-boundary.
     - 2.1.4 *Filter line search*: backtrack from $\alpha^{\max}$ until trial step is filter-acceptable.
     - 2.1.5 Update $(\mathbf{x}, \mathbf{y}, \mathbf{s})$.
   - 2.2 **Decrease** μ (e.g. $\mu \leftarrow \max(0.2\mu, \mu^{1.5})$).

Convergence (convex case): polynomial, $O(\sqrt{n}\log(1/\epsilon))$ outer iterations.

Convergence (nonconvex case): local — to a KKT point under standard assumptions (constraint qualifications).

Roadmap

Coding Break #4 — Same solver, much harder problem

Our 2D LP was convex; the central path was literally a straight line; IPOPT barely broke a sweat.

Now we run IPOPT on **HS071** — a textbook *nonconvex* NLP with:

- A product-of-variables objective.
- A nonconvex product inequality $x_1 x_2 x_3 x_4 \geq 25$.
- A quadratic equality $\sum x_i^2 = 40$.
- Box bounds $1 \leq x_i \leq 5$.

Notebook: `part1_solvers/04_ipopt_pyomo_cyipopt.ipynb`

What to watch for in the log:

- `lg(rg)` column populating — inertia correction is firing.
- `ls` letter codes: `h`, `f`, and possibly `R` (restoration).

The classic Hock–Schittkowski problem (HS071)

$$\min_{x_1,\dots,x_4} \; x_1 x_4 (x_1 + x_2 + x_3) + x_3,$$

$$\text{s.t.} \quad x_1 x_2 x_3 x_4 \geq 25, \qquad x_1^2 + x_2^2 + x_3^2 + x_4^2 = 40,$$

$$1 \leq x_i \leq 5, \qquad i = 1, \dots, 4.$$

Optimum: $\mathbf{x}^\star \approx (1.00, 4.74, 3.82, 1.38)$, $f^\star \approx 17.01$.

```
import pyomo.environ as pyo
m = pyo.ConcreteModel()
m.x = pyo.Var([1,2,3,4], bounds=(1, 5), initialize=1.0)
m.obj = pyo.Objective(
  expr = m.x[1]*m.x[4]*(m.x[1]+m.x[2]+m.x[3]) + m.x[3])
m.c1 = pyo.Constraint(expr = m.x[1]*m.x[2]*m.x[3]*m.x[4] >= 25)
m.c2 = pyo.Constraint(expr = sum(m.x[i]**2 for i in [1,2,3,4]) == 40)

pyo.SolverFactory("ipopt").solve(m, tee=True)
```

Reading the IPOPT iteration log

```
iter    objective    inf_pr   inf_du  lg(mu)  ||d||   lg(rg)  alpha_du  alpha_pr  ls
  0  1.6109693e+01  1.12e+01  5.28e-01  -1.0  0.00e+00    -   0.00e+00  0.00e+00   0
  1  1.6384032e+01  8.37e+00  1.42e+01  -1.0  2.12e+00    -   7.86e-01  2.49e-01  2f
  2  1.7008856e+01  8.37e-03  1.71e+00  -1.0  1.57e+00    -   1.00e+00  1.00e+00h  1
  3  1.6993368e+01  7.50e-04  1.30e-01  -2.5  6.83e-02    -   1.00e+00  9.99e-01h  1
  4  1.7014114e+01  1.15e-05  8.23e-04  -3.8  1.05e-02    -   1.00e+00  1.00e+00h  1
  5  1.7014017e+01  7.06e-09  7.23e-07  -5.7  1.34e-04    -   1.00e+00  1.00e+00h  1
  6  1.7014017e+01  3.05e-14  2.79e-11  -8.6  6.50e-08    -   1.00e+00  1.00e+00h  1
```

One row per outer Newton iteration. Columns decoded:

`iter`	iteration number (iter 0 = starting point, before any step)
`objective`	current value of $f(\mathbf{x}^k)$ — *not* the barrier objective
`inf_pr`	primal infeasibility $\max(\|\mathbf{h}(\mathbf{x})\|_\infty,\ \max_i \max(g_i(\mathbf{x}), 0))$
`inf_du`	dual infeasibility $\|\nabla_{\mathbf{x}}\mathcal{L}\|_\infty$ (the r_1 residual)
`lg(mu)`	$\log_{10}\mu$ — barrier parameter. $-1.0 \to -8.6$ here: four μ updates
$\|d\|$	∞-norm of the Newton search direction $(\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{s})$
`lg(rg)`	$\log_{10}$ of inertia regularization δ added to W; - when $\delta = 0$
`alpha_du`	dual step length (applied to $\mathbf{s}$ update, capped by fraction-to-boundary)
`alpha_pr`	primal step length (applied to $\mathbf{x}$, after filter line search)
`ls`	# of line-search trials; *letter* = step type (see below)

Line-search letter codes (after `alpha_pr`): `f` = f-type (objective-decreasing) step, filter *not* augmented; `h` = h-type (feasibility-decreasing), filter *augmented*; `R` = restoration phase entered; `F`/`H` = f/h-type while in restoration; `s` = accepted by "soft restoration"; `r` = second-order correction used.

Everything visible here comes from the theory we built. The μ decrease $-1.0 \to -2.5 \to -3.8 \to -5.7 \to -8.6$ is the outer loop, and each row is *one* linearized-KKT Newton step. `inf_pr` and `inf_du` are the two residual norms you minimize; once both are below tolerance and μ is small, IPOPT declares optimality.

What IPOPT prints at the end: final summary

After the iteration table, IPOPT prints a diagnostic block and an **exit status**:

```
Number of Iterations....: 6
                                   (scaled)     (unscaled)
Objective...............: 1.7014017e+01  1.7014017e+01
Dual infeasibility......: 2.79e-11       2.79e-11
Constraint violation....: 3.05e-14       3.05e-14
Complementarity.........: 2.51e-09       2.51e-09
Overall NLP error.......: 2.51e-09       2.51e-09

obj/grad/Jac evals.....: 7 / 7 / 7
Lagrangian Hessian evals: 6
Total CPU secs in IPOPT.: 0.012

EXIT: Optimal Solution Found.
```

Decoding each row:

- **Dual infeas.**: $\|\nabla_{\mathbf{x}}\mathcal{L}\|_\infty$ — "is this a KKT point?"
- **Constraint viol.**: $\max(\|\mathbf{h}\|_\infty, \max_i g_i(\mathbf{x})_+)$ — feasibility.
- **Complementarity**: $\|S\Sigma\mathbf{1}\|_\infty$ — inactive multipliers zero?
- **Overall NLP error**: scaled max of the three above; compared to `tol` (default 10^{-8}) to declare convergence.
- **Scaled vs unscaled**: scaled uses IPOPT's internal scaling; report unscaled values in papers.

IPOPT exit messages: not every “exit” is success

The most common `EXIT:` messages:

`Optimal Solution Found`	Scaled NLP error $<$ `tol`. The only true success.
`Solved To Acceptable Level`	Below looser `acceptable_tol`. *Verify residuals manually.*
`Converged to a point of local infeasibility`	Stationary point of $\|\mathbf{h}\|^2$ found but not feasible.
`Maximum Number of Iterations Exceeded`	Hit `max_iter` (default 3000).
`Restoration Phase Failed`	Usually a modeling error or bad scaling.
`Invalid Number in NLP Function`	NaN/Inf in f, g, h or derivatives — callback bug.

The trap. Students routinely read “Solved To Acceptable Level” as success. It’s not. It means `tol` was *not* reached, but `acceptable_tol` (10^{-6} by default) was, in several consecutive iterations. For reported results, always check `Dual infeasibility` and `Constraint violation` manually.

What Pyomo gives back after `solve()`

The iteration log and final summary are IPOPT's *stdout*. Pyomo separately returns a results object and writes the solution onto the model:

```
# IMPORTANT: declare suffix BEFORE solving, or duals won't be imported.
m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)

results = pyo.SolverFactory("ipopt").solve(m, tee=True)

# 1. Termination status (cross-solver, normalized by Pyomo)
print(results.solver.status)                  # 'ok'
print(results.solver.termination_condition)  # 'optimal'

# 2. Solution values --- written onto m.x, m.obj, etc.
for i in [1,2,3,4]:
    print(f"x[{i}] = {pyo.value(m.x[i]):.4f}")
print(f"f(x*) = {pyo.value(m.obj):.4f}")

# 3. Dual multipliers (now populated on the model)
print(m.dual[m.c1])   # multiplier on x1*x2*x3*x4 >= 25
print(m.dual[m.c2])   # multiplier on sum x_i^2 == 40
```

`tee=True` streams IPOPT's stdout to your terminal; without it the iteration log is silent but the results object is identical.

Where the solution lives after `solve()`

What lives where on the model:

- `pyo.value(m.x[i])` — the primal $\mathbf{x}^\star$.
- `pyo.value(m.obj)` — the objective at the optimum.
- `m.dual[c]` — Lagrange multiplier for constraint `c` (y_j for equality, s_i for inequality). *Requires* `m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)` **before** solving.
- `m.rc[v]` — reduced cost / bound multiplier for variable `v`.
- `results.solver.termination_condition` — `optimal`, `infeasible`, `maxIterations`, `error`, ... Always branch on this *before* using the solution.

Sign convention. Pyomo+IPOPT returns duals with the sign of $\mathcal{L} = f + \sum y_j h_j + \sum s_i g_i$. For a $\leq$ constraint, a *negative* multiplier means "tightening the constraint would *worsen* the objective." Different solvers use different conventions — sanity-check on a small example.

What to watch for when reading IPOPT logs

Healthy log
`inf_pr` and `inf_du` decrease smoothly; `lg(mu)` decreases smoothly; `alpha_pr` $= 1.0$ most iterations; `ls` small.

Unhealthy signs

- `inf_pr` plateaus $\Rightarrow$ infeasible or bad scaling.
- Many small `alpha_pr` $\Rightarrow$ line search rejecting; search direction bad.
- `ls` sustained $> 4 \Rightarrow$ filter thrashing.
- `lg(rg)` appears $\Rightarrow$ inertia correction active (nonconvexity).
- "Solved to acceptable level" $\neq$ "Solved". Verify residuals yourself.

We'll come back to the theory behind these quantities — why the outer loop converges, why Newton converges at each μ, and what Mehrotra's trick buys us.

Aside: Pyomo vs. CasADi — choosing a modeling layer

Both front-end IPOPT (and other solvers). They have different sweet spots — neither is strictly better.

Pyomo's strengths.

- True *algebraic modeling language*: `model.x[i,j]` over index sets, constraints as Python functions over those sets. Reads like mathematical notation.
- First-class **MILP / MINLP**: interfaces to Gurobi, CPLEX, CBC, Bonmin, Couenne, SCIP.
- Structured extensions: `pyomo.dae` (differential–algebraic systems), `pyomo.gdp` (generalized disjunctive programming), `mpi-sppy` / PySP (stochastic programming, scenario decomposition).
- Solver-agnostic — swap solvers with one line.

CasADi's strengths.

- **Symbolic AD** is fast and robust for highly nonlinear problems: exact derivatives, optimized expression graph. Function + derivative evaluations typically much faster than Pyomo's (matters when solvers call them thousands of times).
- Built-in **ODE/DAE integrators with sensitivities** (SUNDIALS), composing naturally with the optimizer.
- **C code generation**: export the NLP + derivatives to standalone C. This is why CasADi dominates embedded MPC.
- The **MX graph** lets you embed integrators, NLP solvers, or root-finders as nodes inside a larger problem (bilevel, multi-stage, sensitivity-based formulations).

Rough rule of thumb.

- MILP / MINLP, large structured LP, stochastic programming, algebraic modeling over index sets → **Pyomo**.
- Continuous NLP (especially nonconvex), optimal control, MPC, parameter estimation, ODEs/DAEs in constraints, anything needing speed or hardware deployment → **CasADi**.
- People often use *both*: CasADi for the continuous nonlinear subproblem, Pyomo (or `gurobipy`) for the integer master in a decomposition scheme.

Roadmap

Summary

Today in three sentences

1. **Self-concordance** is the one inequality ($|F'''| \leq 2(F'')^{3/2}$) that makes Newton on a barrier provably polynomial, with complexity controlled by the barrier parameter ν.
2. **Every convex cone** admits a self-concordant barrier — LP ($\nu = n$), SOC ($\nu = 2$), SDP ($\nu = n$), exp/power ($\nu = 3$) — and Cartesian products compose. Modern solvers (ECOS, CLARABEL, SCS, MOSEK) exploit this.
3. **CVXPY** uses DCP to *construct* convexity syntactically, then lifts your problem to a cone program a self-concordant-barrier IPM can solve.

Complexity facts to remember

- Master theorem: $K = O(\sqrt{\nu}\log(1/\varepsilon))$ outer IPM iterations.
- LP: $\nu = n$ variables. SDP: $\nu = n$ matrix side. SOC: $\nu = 2$ per cone (dimension-free!).
- Short-step analysis gives $\sqrt{\nu}$; long-step / Mehrotra gives ν worst case but is typically

Further reading

- **Nesterov & Nemirovski**, *Interior-Point Polynomial Algorithms in Convex Programming* (SIAM 1994) — **the** book. Chapter 2 introduces self-concordance; Ch. 3 the master theorem.
- **Boyd & Vandenberghe**, *Convex Optimization* (2004), Ch. 11 — pedagogically clean 1-page statements of the key results.
- **Renegar**, *A Mathematical View of Interior-Point Methods* (SIAM 2001) — very short, very rigorous.
- **Grant, Boyd, Ye** (2006), "Disciplined Convex Programming" in *Global Optimization* (Springer) — the DCP founding paper.
- **CVXPY documentation**: `https://www.cvxpy.org`. The "Advanced" section covers graph implementations and the solver interface.
- **MOSEK Modeling Cookbook**: `https://docs.mosek.com/modeling-cookbook/` — a catalogue of conic reformulations for every convex function you might want.